

Sadržaj

1.	Uvod	1
2.	Pregled tehnologija i pristupa za obradu događaja u stvarnom vremenu	3
2.1.	Događajno vođena arhitektura i obrasci prijenosa poruka	4
2.1.1	Događaji, vremenske oznake i redoslijed obrade	4
2.1.2	Objavljivanje, pretplata i zahtjev-odgovor u asinkronoj komunikaciji	8
2.1.3.	Redovi poruka i uravnotežavanje opterećenja	9
2.2.	Posrednici poruka i kriteriji odabira	10
2.2.1.	NATS Core arhitektura za asinkrono usmjeravanje poruka bez stanja	11
2.2.2.	Perzistencija tokova podataka i upravljanje stanjem pomoću podsustava JetStream	13
2.2.3.	Usporedba NATS-a s alternativnim rješenjima.....	15
2.3.	Dvosmjerna komunikacija pomoću WebSocketeta i Socket.io protokola.....	16
2.3.1.	WebSocket.....	17
2.3.2.	Socket.io	19
2.4.	Tehnološki stog za implementaciju prototipa.....	20
2.4.1.	Node.js i Express	21
2.4.2.	Python simulatori izvora događaja.....	21
2.4.3.	PostgreSQL za trajnu pohranu	22
2.5.	Sažetak odabira tehnologija u odnosu na zahtjeve rada.....	22
3.	Problem	23
4.	Metodologija i oblikovanje rješenja	26
4.1.	Raspodijeljena komponentna arhitektura s topologijom komunikacijskih predmeta i modelom pohrane	27
4.2.	Model asinkronih tokova događaja od simulatora uređaja i životni ciklus poruke	28
4.3.	Matrica sljedivosti zahtjeva	29
5.	Implementacija	31
5.1.	Simulator uređaja	33

5.2. Poslužiteljska aplikacija.....	36
5.2.1. Pristupna točka za uređaje	36
5.2.2. Sloj obrade i pohrane podataka.....	41
5.2.3. Pristupna točka za korisničko sučelje	43
5.3. Korisničko sučelje	48
5.4. Podatkovni model i trajna pohrana	55
5.5. Pregled načina komunikacije i sudionika.....	57
5.5.1. Pregled NATS predmeta i sudionika	57
5.5.2. Pregled socket.io događaja i sudionika	58
5.5.3. Pregled krajnjih točaka za http zahtjeve	60
5.6. Implementacijska ograničenja	60
6. Evaluacija prototipa	62
7. Zaključak.....	62
Sažetak	67
Summary	68
Dodatak A: Elektronički dodaci i izvorni kod	70
Dodatak B: NATS primjeri koda i način korištenja	71
B.1. Korištenje NATS-a u Node.js okruženju	71
Dodatak C: WebSocket i Socket.io primjeri koda i način korištenja	83
C.1. WebSocket u programskom jeziku Javascript	83
C.2. Socket.io u Node.js okruženju	84
C.3. Socket.io klijent u programskom jeziku Python.....	88

Popis slika

Slika 4.1. Dijagram komponenti sustava s prikazom komunikacijskih puteva.....	28
Slika 4.2. Prikaz puta poruka od uređaja do korisničkog sučelja.....	29

Popis tablica

Tablica 2.1. Taksonomska usporedba i operativne značajke posrednika poruka	15
Tablica 3.1. Prikaz funkcionalnih zahtjeva, načina provjere te kriterija prihvaćanja za izrađeni sustav.....	23
Tablica 3.2. Prikaz nefunkcionalnih zahtjeva, načina provjere te kriterija prihvaćanja za izrađeni sustav.....	25
Tablica 5.1. Komponente sustava, korištene tehnologije i odgovornosti.....	32
Tablica 5.2. Stupci tablice podaci u bazi podataka, njihov tip i uloga.....	56
Tablica 5.3. Pregled NATS predmeta zajedno s kontekstom unutar sustava	58
Tablica 5.4. Pregled Socket.io događaja zajedno s kontekstom unutar sustava.....	59
Tablica 5.5. Pregled HTTP krajnjih točaka zajedno s kontekstom unutar sustava	60

1. Uvod

Obrada događaja u stvarnom vremenu sveprisutan je problem današnjeg računarstva i programskog inženjerstva. S porastom složenosti modernih računalnih sustava, raste i potrošnja računalnih resursa, a time i potreba za razvojem adekvatnih programskih rješenja koje će pružiti potrebne karakteristike i mogućnosti korisniku. U distribuiranim informacijskim sustavima sve veći broj uređaja, korisničkih sučelja i poslužiteljskih komponenti ne razmjenjuje podatke samo povremenim zahtjevima, nego kontinuirano proizvodi događaje koji moraju biti prihvaćeni, obrađeni, pohranjeni i prikazani uz što manju latenciju. U klasičnim klijent-poslužiteljskim sustavima komunikacija se često temelji na obrascu zahtjeva i odgovora. Takav pristup može biti dovoljan za povremeni dohvat podataka, ali postaje ograničavajući u slučajevima u kojima se promjene stanja pojavljuju nepredvidivo i velikom učestalošću. Obrada događaja u stvarnom ili blizu-stvarnom vremenu često podrazumijeva pristizanje velikog volumena podataka u kratkom vremenu, što čini njihovo učinkovito primanje i obradu posebno važnom.

Postojeći pristupi i alati za obradu događaja omogućuju različite oblike asinkrone komunikacije, međuspremanja i raspodjele opterećenja u kojem niska latencija, visoka propusnost i skalabilnost igraju najvažniju ulogu. Osiguravanje pouzdanosti sustava i u slučaju gubitka veze te sprječavanje gubitka podataka također su izazov u rješavanju navedenih problema.

Cilj rada je oblikovati, implementirati i empirijski provjeriti modularni prototip koji eliminira spomenute nedostatke, osigurava navedene komunikacijske zahtjeve te implementira rješenje za navedene probleme komunikacije i primanja podataka u stvarnom vremenu. Izgrađeni prototip sadržava poslužitelj koji obrađuje podatke pristigle s većeg broja uređaja te korisničku aplikaciju koja omogućuje praćenje stanja uređaja te prikaz primljenih podataka. Također, funkcionalnost prototipa ispituje se programskim simulatorima uređaja, pokrenutima u većem broju instanci, koji, za potrebe ispitivanja i provjere ispunjenosti zahtjeva, generiraju podatke i šalju ih poslužitelju na obradu. Potrebno je ispitati ponašanje poslužitelja pri različitim opterećenjima i rubnim slučajevima te provjeriti njegovu pouzdanost u svakom od navedenih

scenarija. Od sustava se očekuje robusnost i primjereno ponašanje i u slučaju mrežnih oštećenja, kao što je slučaj privremenog gubitka veze te u slučajevima kratkotrajnih skokova mrežnog prometa i volumena pristiglih podataka (engl. *burst loads*). Uspješnost izrade sustava je provjerena upravo analizom pouzdanosti, latencije i robusnosti u svim navedenim slučajevima i uvjetima.

Rješenje se temelji na događajno vođenoj arhitekturi i kombinaciji više programskih tehnologija. Simulatori izvora događaja izvedeni su u programskom jeziku Python, poslužiteljski dijelovi sustava implementirani su u okruženju Node.js, komunikacija između poslužiteljskih komponenti ostvaruje se pomoću NATS posrednika poruka, dok se dvosmjerna komunikacija s uređajima i korisničkim sučeljem ostvaruje pomoću Socket.io protokola. Za trajnu pohranu događaja koristi se PostgreSQL baza podataka. U takvoj arhitekturi NATS predmeti služe za razdvajanje proizvođača i potrošača poruka, Socket.io omogućuje prijenos događaja prema povezanim klijentima, a baza podataka osigurava naknadnu provjeru i prikaz povijesnih zapisa.

Glavni programski artefakti rada obuhvaćaju simulator izvora podataka, pristupnu točku za uređaje, sloj za obradu i pohranu događaja, pristupnu točku za korisničko sučelje, web sučelje za nadzor događaja te podatkovni model za trajnu pohranu zapisa. Svaki generirani događaj sadrži identifikator uređaja, jedinstveni identifikator podatka, vrijednost podatka i vremensku oznaku nastanka događaja. Takva struktura omogućuje razlikovanje vremena generiranja, vremena prikaza i vremena pohrane, čime se stvaraju pretpostavke za mjerenje latencije i provjeru ispravnosti toka obrade. U okviru gamifikacijskog proširenja isti tok događaja može se koristiti kao ulaz za pravila dodjele bodova, evidenciju korisničke aktivnosti i prikaz rezultata u korisničkom sučelju.

Rad je organiziran na sljedeći način. U drugom poglavlju obrađuju se srodni pristupi i tehnološki okvir za obradu događaja u distribuiranim sustavima, s naglaskom na događajno vođenu arhitekturu, komunikacijske obrasce, posrednike poruka, NATS, WebSocket i Socket.io. U trećem poglavlju formaliziraju se funkcionalni i nefunkcionalni zahtjevi prototipa, struktura događaja, uvjeti rada i mjerne veličine za procjenu odziva, propusnosti i pouzdanosti. Četvrto poglavlje opisuje razvijeni prototip na visokoj razini te pruža pregled strukture programskog rješenja opisno i putem dijagrama. Peto poglavlje opisuje detaljno svaku

komponentu razvojenog programskog rješenja, uz primjere koda koji prikazuju funkcionalnost. Zaključno poglavlje sažima ostvarene rezultate, razmatra ograničenja razvijenog rješenja i opisuje mogućnosti poboljšanja, kao i ciljeve budućeg rada.

2. Pregled tehnologija i pristupa za obradu događaja u stvarnom vremenu

U ovom poglavlju razmatraju se pristupi i tehnologije relevantni za oblikovanje modularnog sustava za obradu događaja u blizu stvarnom vremenu. Naglasak nije na iscrpnom pregledu svih dostupnih alata, nego na konceptima koji izravno utječu na arhitekturu razvijenog prototipa: događajno vođenu komunikaciju, asinkroni prijenos poruka, posrednike poruka, dvosmjernu komunikaciju s korisničkim sučeljem i trajnu pohranu događaja. Razmatrani pristupi služe kao osnova za kasnije definiranje zahtjeva, oblikovanje arhitekture i odabir mjerila za provjeru rada sustava.

Obrada događaja u stvarnom vremenu područje je koje podrazumijeva specifične zahtjeve koji se razlikuju od zahtjeva uobičajenih informacijskih sustava. Naime, u ovakvim sustavima prvenstveno postoji potreba za mogućnošću obrade velikog broja događaja koji pristižu velikom brzinom, često i istovremeno, s velikog broja različitih lokacija. Potrebno je smanjiti mogućnost preopterećenja poslužitelja te gubitka događaja. Osim toga, u ovakvim slučajevima, osiguravanje niske latencije često je izazov, a nužno je za postizanje željenog korisničkog iskustva. Kao rješenje navedenih problema, često se koristi distribuirana arhitektura koja uključuje više odvojenih čvorova (engl. *nodes*), koji međusobno dijele posao. Ovime se smanjuje opterećenje svakog pojedinog čvora te je manja vjerojatnost pojave zagušenja ili preopterećenja jednog čvora (engl. *bottleneck*), koje bi dovelo do prestanka ili narušavanja kvalitete rada sustava. Ovakav pristup omogućuje i nadogradnju sustava po potrebi horizontalnim skaliranjem, dodavanjem većeg broja čvorova, bez potrebe za promjenom arhitekture. Distribuirani sustavi imaju i svoje nedostatke, posebno glede povećanja složenosti

i troškova održavanja. Međutim, prednosti ovakvih sustava, u kontekstu sustava za obradu događaja u stvarnom vremenu, uglavnom opravdavaju nedostatke.

Kako bi distribuirani sustav radio na željeni način, čvorovi moraju imati mogućnost komunikacije i uspostave pouzdane veze, kako međusobno, tako i s klijentima, fizičkim uređajima i ostalim odvojenim komponentama sustava. Također, važno je i postojanje mehanizama oporavka i redundancije, kojima se omogućava nastavak rada i u slučaju kvara određenog dijela sustava ili prekida veze između određenih komponenti. Postoje različiti pristupi izradi distribuiranih sustava, od kojih svaki ima prednosti i mane. Ne postoji univerzalno dobro ili superiorno rješenje. Odabir arhitekture i načina implementacije ovisi o konkretnom slučaju i prirodi problema, a u svakom slučaju potrebno je napraviti određene kompromise. Jedan od modela koji opisuju navedeno je tzv. CAP teorem (*Consistency, Availability, Partition tolerance*) [1], koji kaže da nije moguće osigurati visoku konzistentnost, dostupnost i toleranciju na particioniranje istovremeno niti u jednom distribuiranom sustavu. Moguće je odabrati dvije od navedene tri karakteristike, dok će treća biti kompromitirana.

U nastavku poglavlja razmatraju se događajno vođeni obrasci, posrednici poruka, dvosmjerna komunikacija i tehnologije korištene u izgradni prototipa.

2.1. Događajno vođena arhitektura i obrasci prijenosa poruka

Distribuirani sustavi često su izgrađeni na način da koriste arhitekturu temeljenu na događajima (engl. *event driven*). Ovo je arhitekturni odabir koji se razlikuje od uobičajenog komunikacijskog obrasca zahtjeva i odgovora, utoliko što se omogućuje pasivno primanje podataka u bilo kojem trenutku, bez potrebe za slanjem zahtjeva [2].

2.1.1 Događaji, vremenske oznake i redoslijed obrade

Događaj (engl. *event*) predstavlja određeni podatak ili skup podataka koji komponenta izaziva i o čemu, po potrebi, obavještava ostale komponente. Događaj može predstavljati različita zbivanja i promjene stanja u sustavu. Npr. pristizanje određenih podataka, dovršenost obrade, povezivanje novog uređaja, korisnički zahtjev, itd. Svaki događaj je, po definiciji, nepromjenjiv i neopoziv. [3] Nije moguće poništiti već izazvani događaj u sustavu pa se pogreške ispravljaju novim događajem, a ne modifikacijom postojećeg. Osim toga, svaki događaj ima svoju vremensku oznaku (engl. *timestamp*), koja predstavlja trenutak u kojem se događaj zbio, što je ključno za sinkronizaciju i pravovremeni rad svih dijelova sustava. Međutim, unatoč postojanju vremenskih oznaka, određivanje redoslijeda događaja nije uvijek trivijalno. Naime, u sustavu često ne postoji univerzalni sat, već svaki čvor samostalno mjeri vrijeme i pripisuje vremenske oznake svojim događajima. Nadalje, razlikuje se vrijeme samog zbivanja događaja (engl. *event time*) te vremenska oznaka trenutka u kojem je sustav obradio događaj (engl. *processing time*). [4] Razlikovanje ovih koncepata je važno u sustavima koji su dizajnirani za obradu događaja redom kojim su se događaji zbivali, a ne nužno redom pristizanja. Naime, u uvjetima u kojima je komunikacijska veza spora, može doći do kašnjenja i uzrokovati redoslijed pristizanja događaja koji ne odgovara stvarnom redoslijedu zbivanja događaja. U ovakvim slučajevima, sustav mora, ako je to potrebno, uzeti u obzir stvarni redoslijed.

S obzirom da univerzalni sat u sustavu ne postoji, pojavljuje se izazov određivanja točnog redoslijeda događaja. Za mnoge aktivnosti u distribuiranom sustavu potrebno je sa sigurnošću utvrditi koji događaj se zbio prije. Jedan od čestih načina rješavanja navedenog problema je korištenje logičkih satova, kao u npr. Lamportovom algoritmu, koji efektivno podrazumijevaju brojač događaja na nekom čvoru distribuiranog sustava. [5] Logički sat podrazumijeva vrijeme predstavljeno cijelim brojem, koji se inkrementira pri svakom novom događaju te se šalje ostalim čvorovima zajedno s ostalim podacima u svrhu sinkronizacije. Nakon primanja podatka o logičkom satu čvora pošiljatelja, primatelj ažurira svoj logički sat postavljanjem njegove vrijednosti na vrijednost za jedan veću od vrijednosti $\max(\text{logički_sat_primatelja}, \text{logički_sat_pošiljatelja})$, kako bi se sa sigurnošću uspostavio kronološki redoslijed. Iako logički satovi ne pružaju informaciju o apsolutnom vremenu događaja, oni uspostavljaju relaciju koja sa sigurnošću utvrđuje da određeni događaj prethodi nekom drugom događaju, što je za mnoge primjene dovoljno.

U distribuiranim sustavima važno je smanjiti mogućnost gubitka poruka, a ovisno o konkretnom rješavanom problemu, rizik gubitka poruka može biti više ili manje prihvatljiv. U skladu s tim, postoje različite razine osiguravanja primanja poruke, koje se implementiraju sukladno potrebama i dostupnim resursima. Te razine su : najviše jednom (engl. *at-most-once*), barem jednom (engl. *at-least-once*) te točno jednom (engl. *exactly-once*). [6] Pristup najviše jednom podrazumijeva slanje poruke bez ikakvih naknadnih radnji, kojima bi se osiguralo primanje poruke. Ovakav pristup je jednostavan i jeftin, no riskira gubitak poruke u slučaju prekida veze ili kvara na uređaju primatelju. Pristup barem jednom radi na principu ponovljenog slanja poruke sve do pristizanja potvrde o primanju. Dok potvrda ne stigne, pošiljalatelj ponavlja slanje iste poruke u određenim intervalima. Ovime se znatno smanjuje mogućnost gubitka poruke, no povećava se mogućnost pojave dvostrukih poruka na primatelju. Naime, zbog mrežne latencije, moguće je da primatelj dobije poruku i pošalje potvrdu, no za vrijeme putovanja potvrde mrežom, pošiljalatelj već pošalje novu kopiju poruke. Za mnoge namjene, pristizanje dvostrukih poruka nije velik nedostatak pa ovaj pristup nudi povoljne karakteristike. Najsofisticiraniji pristup, točno jednom, ujedno vodi i ka najvećem usložnjavanju arhitekture, no osigurava zaštitu od dvostrukih poruka i gubitka poruke istovremeno. Ovo je, u praksi, teško postići, no efektivno je moguće dobiti takav rezultat implementacijom deduplikacijske logike na čvorovima primateljima, uz istodobnu primjenu pristupa barem jednom. Deduplikacijska logika filtrira duplicirane poruke te time sprječava ponavljanje već obavljenih radnji. Drugi uobičajeni pristup je primjena idempotentnih operacija na čvoru primatelju. To su operacije koje, unatoč izvršavanju više puta, samo jednom mijenjaju stanje sustava. U tom slučaju, obrada iste poruke više puta neće imati negativan utjecaj.

Jedna od glavnih svrha distribuiranih sustava je smanjenje opterećenja na svakom pojedinom čvoru, kako bi ukupni rezultat bio brža izvedba s manjom mogućnošću preopterećenja bilo kojeg konkretnog čvora. Važnu ulogu u ovome igra koncept particioniranja (engl. *sharding*), što podrazumijeva raspodjelu većeg zadatka na više manjih te njihovu raspodjelu po čvorovima sustava. [7] Ovime se paralelizira izvođenje zadatka, čiji su dijelovi dovoljno neovisni da se nesmetano mogu izvoditi na različitim lokacijama. Ovakav pristup neizostavan je dio rješavanja brojnih vrsta problema, posebno onih u kojima se rezultati različitih čvorova po završetku prikupljaju, agregiraju te obrađuju u završnoj fazi. Još jedna karakteristika distribuiranih sustava je redundancija. Ona se može odnositi na redundanciju

podataka, ali i na arhitekturnu redundanciju računalnih resursa. Naime, ako sustav raspolaze određenim podacima potrebnim za rad, uglavnom nije prihvatljivo da se ti podaci nalaze isključivo na jednom čvoru, jer u slučaju kvara na čvoru ili gubitka veze s njim, sustav nema pristup podacima te prestaje s radom. Redundancija podataka podrazumijeva raspodjelu podataka na više čvorova, tako da se kopije istog podatka nalaze na više različitih čvorova. Postoje različiti stupnjevi redundancije. Nije uvijek nužno imati kopije svih postojećih podataka, a također, nije potrebno da se svi podaci nalaze na svim čvorovima sustava. Ovakav pristup bi, dakako, bio najsigurniji i najpouzdaniji, no zbog resursnih ograničenja u vidu kapaciteta pohrane, rijetko se koristi. Obično se koristi određena sredina, koja pruža zadovoljavajuću pouzdanost, bez nepotrebnih troškova. Redundancija arhitekture podrazumijeva sposobnost sustava da nastavi s radom i održi funkcionalnost u slučaju kvara dijela sustava. Ovo se postiže izbjegavanjem uvođenja točke pada (engl. *single point of failure*). [8] U praksi, to je obično središnji čvor na koji su ostali čvorovi povezani i s kojim svi komuniciraju. U slučaju takve arhitekture, nužni su mehanizmi oporavka pa čak i kopija središnjeg čvora, pomoću koje rad sustava može nastaviti.

Prilikom dizajniranja distribuiranih sustava, potrebno je uzeti u obzir i heterogenost sustava, tj. različitost čvorova u sustavu. Konkretno, česta je pojava da različiti čvorovi sustava imaju različite mogućnosti, različitu količinu računalnih resursa pa čak i različit operacijski sustav. Ovo direktno utječe na njihovu mogućnost obrade događaja i primanja podataka te brzinu obavljanja radnji. Čest problem je neusklađenost brzine pristizanja podataka koje pošiljalac šalje sa brzinom obrade istih na čvoru primatelju. Npr. moguće je da, kad bi pošiljalac slao događaje određenom brzinom, primatelj ne bi bio u stanju pratiti taj tempo zbog svojih ograničenih mogućnosti. Načini rješavanja ovakvog problema su, već spomenuta raspodjela zadataka na više čvorova, ali i implementiranje specifičnih komunikacijskih obrazaca karakterističnih za distribuirane sustave, koji često pružaju veću kontrolu nad tempom primanja i obrade podataka.

Razlikovanje vremena nastanka događaja i vremena njegove obrade važno je za sustave u kojima se procjenjuje odziv ili kašnjenje obrade. Vremenska oznaka nastanka omogućuje praćenje trenutka u kojem je događaj proizveden, dok vremenske oznake kasnijih faza obrade omogućuju procjenu latencije kroz komunikacijski i obradni tok.

2.1.2 Objavljivanje, pretplata i zahtjev-odgovor u asinkronoj komunikaciji

Zbog navedenih karakteristika distribuiranih sustava, uvode se komunikacijski obrasci prilagođeni uporabi u takvom okruženju. Iako distribuirani sustavi koriste i uobičajene oblike komunikacije, takav pristup ne osigurava uvijek dovoljnu efikasnost i sinkroniziranost rada sustava. Nadalje, ovakav pristup odstupa od arhitekture temeljene na događajima, kakva se pokazuje boljom za upotrebu u distribuiranim sustavima. Neki od tipičnih korištenih obrazaca, podržanih u mnogim alatima i bibliotekama namijenjenim ostvarivanju komunikacije između komponenti su objava-pretplata (engl. *publish-subscribe*), zahtjev-odgovor (engl. *request-response*) te komunikacija redovima poruke (engl. *message queues*) [9].

Komunikacija objavljivanjem i pretplatama obrazac je koji se temelji na definiranju određenih predmeta (engl. *subject*) na koje svaka komponenta može poslati podatak ili poruku. Prilikom slanja poruke, komponenta pošiljatelj bira predmet na koji poruku šalje, uglavnom na temelju vrste, značenja i uloge navedene poruke u sustavu. Ostale komponente kojima su navedene poruke namijenjene, pretplaćuju se na predmete na kojima očekuju da će poruke biti objavljene. Ovime se ostvaruje arhitektura konceptualno slična forumu, gdje svaka komponenta, pretplatom na odgovarajući predmet, prima njoj potrebne poruke, bez opterećenja porukama irelevantnim za njezin rad. Ovakav sustav također omogućava pošiljatelju da poruku pošalje samo jednom, činom objavljivanja na temu, a poruka automatski pristiže svim pretplaćenim komponentama. Ovime se izbjegava opetovano slanje iste poruke više različitim primatelja te se štede vrijeme i mrežni resursi. Predmeti se često definiraju hijerarhijski, pri čemu se slojevi hijerarhije odvajaju znakom točke. Ovakav logičan sustav nazivlja predmeta omogućava, uz jednostavnije održavanje i programiranje, i pretplatu na više tema istovremeno uporabom specijalnih znakova koji zamjenjuju jedan ili više dijelova nomenklature hijerarhije (engl. *wildcard*). To omogućuje primanje poruka s cijelog skupa povezanih predmeta.

Uz obrazac objave i pretplate često se koristi i obrazac zahtjeva i odgovora. Komunikacija principom zahtjev-odgovor, klasičan je oblik komunikacije koji se viđa u gotovo svim računalnim sustavima. Temelji se na slanju zahtjeva određenoj komponenti, nakon čega se očekuje odgovor u kojem se nalaze tražene informacije. Ovaj pristup vidljiv je u HTTP

protokolu, gdje se koristi za pozivanje API funkcija s poslužitelja, no sam po sebi, nije naročito prikladan za distribuirane sustave. Ipak, unutar alata i okruženja za rad s distribuiranim sustavima, ovaj se pristup često kombinira s već spomenutim sustavom objava i pretplata. Naime, zahtjev je moguće također objaviti na željeni predmet, baš kao i svaku drugu poruku. Kod objave tipične poruke, nema provjere uspješnosti obrade niti potvrde primitka. Međutim, objava zahtjeva podrazumijeva da se odgovor očekuje. Implementacija navedenoga varira u ovisnosti o radnom okviru i programskom jeziku, no čest pristup je korištenje obećanja (engl. *Promise*) te postavljanja vremenskog intervala nakon kojeg se javlja pogreška (engl. *timeout*).

Obrasci objave i pretplate te zahtjeva i odgovora prikladni su za modularne sustave u kojima se želi smanjiti izravna ovisnost između proizvođača i potrošača poruka. Komponente u takvom modelu razmjenjuju poruke preko dogovorenih komunikacijskih predmeta, bez potrebe za poznavanjem unutarnje izvedbe drugih komponenti. Time se olakšava razdvajanje prihvata, obrade, pohrane i prikaza događaja, što je osobito važno u sustavima s niskom latencijom i mogućnošću kasnijeg proširenja.

2.1.3. Redovi poruka i uravnotežavanje opterećenja

Redovi poruka (engl. *message queues*), tip su komunikacija koji koristi međuspremnik u kojem se poruke nalaze sve dok ih primatelj ne obradi. Ovakav pristup je koristan u slučajevima u kojima primatelj svojom brzinom obrade podataka ne prati nužno brzinu kojom ih pošiljalac šalje. Sve dok primatelj nije spreman pročitati i obraditi poruku, ona se nalazi u međuspremniku, što omogućuje asinkronost čvorova u sustavu. Većina implementacija se temelji na destruktivnom čitanju, tj. karakteristici međuspremnika koja nalaže da, kad komponenta pročita poruku, poruka se briše. Komponenta čita prvu poruku iz reda, a njenim brisanjem osigurava se protok podataka te eventualno pristizanje i obrada ostalih poruka. Činjenica da se poruke po čitanju brišu, omogućava korištenje redova poruka za ujednačavanje opterećenja. Naime, ako više komponenti primatelja čita iz istog reda poruka, svaku poruku primit će samo jedna komponenta. Ovo odgovara konceptu raspodjele zadataka po čvorovima te njihovoj paralelizaciji radi bržeg izvršavanja. Postoji nekoliko pristupa ujednačavanju

opterećenja. U relativno homogenim sustavima, u kojima komponente imaju podjednake mogućnosti i resurse te su zadaci podjednako zahtjevni, često je kružno raspoređivanje poruka (engl. *round robin*). [10] Komponente čitaju poruke iz reda jedna po jedna, a nakon što su sve pročitale, čitanje kreće ponovo od prve komponente. Ovako svaka komponenta dobiva podjednak broj zadataka koje izvršava. Ako zadaci značajno variraju u složenosti ili ako sustav posjeduje visok stupanj heterogenosti, primjeren odabir je dinamičko raspoređivanje zadataka na temelju brzine obrade. U ovom pristupu, na početku svaka komponenta preuzima jedan zadatak iz reda poruka, obrađuje ga, a po završetku, preuzima idući zadatak, neovisno o statusu i napretku ostalih komponenti. Ovime se postiže visoka asinkronost i maksimalno iskorištenje resursa, jer brže komponente ujedno uspijevaju obaviti veći broj zadataka, a one sporije prirodno dobivaju manji broj zadataka te se na njima ne čeka.

Osim klasične implementacije redova poruka u kojoj se svaka poruka čita samo jednom, neovisno o broju komponenti, postoje i varijante u kojima svaka komponenta dobiva po jednom svaku poruku. Ovakva izvedba najčešće se postiže uporabom potrošača (engl. *consumer*), koji predstavlja poziciju u redu poruka s koje se poruka čita. Sam red poruka, u ovom slučaju, zadržava poruke i nakon čitanja kako bi ostale komponente imale pristup istima, što odskake od stroge definicije reda poruke, no iz perspektive svake pojedine komponente, stvara se privid destruktivnog čitanja.

Redovi poruka i grupe potrošača omogućuju raspodjelu obrade među više potrošačkih komponenti. Takav pristup posebno je koristan kada brzina pristizanja događaja može premašiti brzinu obrade pojedinačne komponente. U sustavima s promjenjivim opterećenjem time se smanjuje rizik stvaranja uskog grla i omogućuje postupno skaliranje obradnog sloja.

2.2. Posrednici poruka i kriteriji odabira

Kako bi se opisani mehanizmi razmjene poruka ostvarili u sustavu, uobičajeno je korištenje posrednika poruka (engl. *message broker*). Posrednik poruka jest programska komponenta koja prima, usmjerava i prosljeđuje poruke između drugih komponenti sustava. U događajno vođenim sustavima posrednik poruka omogućuje da proizvođači događaja i

potrošači događaja ne ovise jedni o drugima izravno. Proizvođač poruke ne mora poznavati mrežnu adresu ni unutarnju izvedbu potrošača, nego poruku objavljuje preko posrednika. Potrošač, s druge strane, prima samo one poruke za koje je iskazao interes pretplatom ili drugim komunikacijskim mehanizmom. Njihova uloga očituje se u labavom povezivanju (engl. *loose coupling*) i asinkronoj komunikaciji između pojedinih komponenti programske potpore. To je odvojena komponenta čija je uloga primanje poruka namijenjenih ostalim komponentama te provedba spomenutih algoritama i načela njihovog slanja pravovremenim prosljeđivanjem. Posrednici poruka implementiraju logiku pretplata, objava, predmeta i sl. Ovisno o složenosti samog sustava te sofisticiranosti zahtjeva, postoje brojna rješenja koja variraju po složenosti, npr. Kafka, Redis, NATS, RabbitMQ i sl. Ovo poglavlje fokusira se na prikaz osnovnih funkcionalnosti NATS-a, kao alata korištenog za implementaciju prototipa u sklopu ovog rada, pružajući opis njegovih funkcionalnosti te usporedbu s alternativama.

Posrednik poruka omogućuje smanjenje izravne ovisnosti između proizvođača i potrošača događaja. Umjesto da komponente međusobno pozivaju funkcije ili izravno poznaju mrežne adrese drugih komponenti, poruke se razmjenjuju preko zajedničkog komunikacijskog sloja. Takav pristup olakšava modularno oblikovanje sustava jer se pojedine komponente mogu dodavati, uklanjati ili mijenjati uz manji utjecaj na ostatak sustava.[11]

2.2.1. NATS Core arhitektura za asinkrono usmjeravanje poruka bez stanja

NATS je sustav namijenjen komunikaciji među komponentama distribuiranih sustava. Radi se o projektu otvorenog koda (engl. *open source*) koji se nalazi pod *Cloud Native Computing Foundation* (CNCF) organizacijom kao inkubirajući (engl. *incubating*) projekt [12]. To je druga razina zrelosti projekata, prema CNCF definiciji, zrelije od pješčaničke razine (engl. *sandbox*), a manje zrelo od diplomirane razine (engl. *graduated*). Odlikuju ga jednostavnost, visoke performanse, niska latencija, skalabilnost i visoka propusnost. NATS-ov server napisan je u programskom jeziku Go, što doprinosi njegovom brzom pokretanju [13]. Središnji dio NATS ekosustava predstavlja središnji NATS skup funkcionalnosti, NATS Core [14]. NATS Core omogućuje osnovnu funkcionalnost razmjene poruka po principu „pošalji i zaboravi“ (engl. *fire-and-forget*). Osnovna i najčešće korištena NATS funkcionalnost je razmjena poruke

putem objave i pretplate (engl. *publish-subscribe*). Ovaj način komunikacije zamjenjuje klasičan, no manje modularan i adaptibilan sustav direktne komunikacije više komponenti. Naime, prilikom slanja poruke, komponenta definira predmet (engl. *subject*) kojoj je odaslana poruka pridružena. Drugom riječima, poruka se šalje na određeni predmet, slično kao što se objave na forumu objavljuju upravo pod određenom temom. Druge komponente sustava se mogu pretplatiti na predmete za koje se zna da će sadržavati njima relevantne podatke. Kad se komponenta pretplati na predmet, dobivat će sve podatke koje je neka druga komponenta objavila na taj predmet. Komponente dobivaju poruke po principu guranja (engl. *push*), što znači da NATS poslužitelj automatski predaje poruku pretplaćenju komponenti, bez potrebe da komponenta šalje dodatne zahtjeve. Ovo može predstavljati nedostatak jer pretplaćena komponenta ne odlučuje sama kada će i kojom brzinom primiti poruke. U slučaju brzine pristizanja novih poruka koja je prevelika da bi je komponenta mogla pratiti, može doći do preopterećenja komponente i gubitka poruke. Komponenta koja ne može pratiti brzinu pristizanja podataka zove se spori potrošač (engl. *slow consumer*). Jedna od osnovnih karakteristika NATS-a je činjenica da sve pretplaćene komponente dobivaju poruke koje pristizu. Drugim riječima, ako je više komponenti pretplaćeno na isti predmet, a na taj predmet pristigne jedna poruka, čitanje poruke od strane jedne komponente ne onemogućava ostalim komponentama dobivanje potrebnih podataka. Po ovome se NATS komunikacija razlikuje od oblika komunikacije, kao što su redovi poruka (engl. *message queues*) i cjevovoda (engl. *pipelines*), gdje samo jedan potrošač dobiva i obrađuje poruku. Međutim, NATS u svom izvornom obliku ne podržava trajnost podataka (engl. *persistence*). Jednom kad je poruka poslana svim pretplaćenim komponentama, ona se trajno briše s poslužitelja. To znači da, ako se neka komponenta naknadno pretplati na isti predmet, neće dobiti prethodno poslani poruke. Također, ako se poruka pošalje na predmet bez pretplatnika, poruka je uzalud poslana te ne ostaje pohranjena na poslužitelju. U svrhu rješavanja ovog problema, postoje nadogradnje opisane u sljedećim poglavljima.

Osim osnovnog sustava objava i pretplata, NATS nudi i mogućnost komunikacije zahtjev-odgovor (engl. *request-reply*). Poruke zahtjev-odgovor također se objavljuju na određeni predmet, no komponenta koja ih prima na njih odgovara (engl. *respond*) slanjem poruke na povratni predmet (engl. *reply subject*). Ovo je korisno u slučajevima kad komponenta treba određene podatke u određenom trenutku, a njihovo pristizanje u nasumičnom trenutku

nije prohvratljivo rješenje. Ovako komponenta samostalno šalje zahtjev u trenutku u kojem su joj podaci potrebni.

U sklopu NATS Core-a postoji i mogućnost korištenja varijante komunikacije u kojoj samo jedna pretplaćena komponenta dobiva poruku. Ovo je pristup koji se temelji na konceptu redova poruka, a ostvaren je korištenjem grupa reda (engl. *queue groups*). Komponente pretplatnici mogu formirati grupe, unutar kojih će samo jedna nasumična komponenta dobiti pristiglu poruku. Drugim riječima, za svaku poruku odabire se jedan primatelj koji će ju primiti, dok ostale komponente tu poruku ne dobivaju. Ovaj pristup je prikladan i efikasan za ostvarivanje uravnotežavanja opterećenja (engl. *load balancing*) te paralelizacije obavljanja posla. Ako se veći i resursno/vremenski zahtjevniji zadatak može rastaviti na više nezavisnih dijelova, izvođenje takvog zadatka značajno se ubrzava ovakvim pristupom. Svaka komponenta preuzima dio zadatka, rješava ga te se na kraju rezultati prikupljaju.

2.2.2. Perzistencija tokova podataka i upravljanje stanjem pomoću podsustava JetStream

JetStream je nadogradnja koja pruža mnoga poboljšanja i dodatna rješenja u odnosu na osnovni NATS [15]. Jedan od osnovnih problema koje Jetstream rješava je nedostatak trajne pohrane poruka u Core NATS-u. Već je spomenuto kako komponente pretplaćene na predmet primaju samo one poruke koje su pristigle za vrijeme pretplate te nema mogućnosti primanja starih poruka jer se one trajno brišu i nestaju s poslužitelja. Jetstream rješava ovaj problem uvođenjem funkcionalnosti trajnosti ili perzistencije podataka. To se ostvaruje uvođenjem tokova (engl. *stream*). Tok je struktura podataka u kojoj se podatci trajno zadržavaju, tj. do isteka zadanog vremenskog ograničenja, a čitaju korištenjem potrošača/konzumera (engl. *consumer*). Prilikom kreiranja toka, definiraju se njegovi parametri, poput maksimalnog vremena trajanja poruka, maksimalne veličine, maksimalnog broja poruka i sl. Moguće je odabrati i tip pohrane koji uvjetuje stupanj trajnosti podataka u toku. Ako se odabere datotečni tip pohrane (engl. *file*), podaci su pohranjeni trajno, sve do isteka njihovog roka definiranog unaprijed postavljenim pravilima. Drugim riječima, podaci se zapisuju na disk te su dostupni i nakon ponovnog pokretanja poslužitelja. Ako se odabere memorijski tip pohrane, podaci se

brišu resetiranjem, što čini ovaj tip pohrane sličnim radnoj memoriji računala, tj. brži, no manje pouzdan. Konzumer se može zamisliti kao pokazivač koji pokazuje na element toka koji određena komponenta trenutno čita. Nakon čitanja elementa, pokazivač se pomiče za jedno mjesto udesno na idući element pa će sljedećim čitanjem biti pročitani novi element. Važna karakteristika ovog pristupa je da se prethodno pročitani elementi ne brišu iz toka podataka. Oni ostaju u njemu trajno pohranjeni. Pomicanje pokazivača ostvaruje privid brisanja, no ako druga komponenta odluči čitati podatke istog toka, koristeći novi konzumer, ona će, potpuno neovisno, moći čitati podatke počevši otpočetak, jer njen pokazivač nije ovisan o pokazivaču prve komponente.

Ponašanje konzumerskog pokazivača detaljnije se može specificirati prilikom kreiranja istog. Tome služe parametri koji se predaju prilikom kreiranja tog objekta. Tim parametrima se određuju karakteristike poput početne pozicije čitanja iz toka. To može uključivati sve poruke toka, uključujući i stare, samo nove poruke ili poruke nakon nekog točno definiranog vremena pristizanja. Moguće je definirati i je li potrebno potvrditi primanje poruke (engl. *acknowledge*). Ovim pristupom povećava se složenost, ali i pouzdanost sustava. Ako se primanje poruke mora potvrditi, postoji i opcija ponovnog slanja poruke određeni broj puta s vremenskim odmakom, dok se primanje ne potvrdi.

Jetstream nudi i mogućnost pohrane parova ključ-vrijednost (engl. *key-value*), slično kao u rječnicima (engl. *dictionary*). Ovo je izuzetno korisno kada više komponenti treba biti u mogućnosti pristupiti dijeljenim podacima, npr. konfiguracijskim parametrima. Takvim pristupom se olakšava dijeljenje istih bez potrebe za direktnom komunikacijom komponenti. Ova funkcionalnost također je izgrađena nad tokovima.

Jedna od naprednijih funkcionalnosti Jetstreama je i mogućnost pohrane većih datoteka, koju pruža *Object Store*. To podrazumijeva različite složenije tipove podataka, poput slika, pdf datoteka i sl. Prijenos datoteka izvršava se dijeljenjem datoteka u komade (engl. *chunking*), čime se omogućuje njihov efikasan i siguran prijenos. Jetstream podržava prijenos, dohvaćanje te brisanje datoteka/objekata s poslužitelja.

2.2.3. Usporedba NATS-a s alternativnim rješenjima

Posrednici poruka značajno se razlikuju prema internoj arhitekturi, memorijskom otisku, mehanizmima upravljanja stanjem te prikladnosti za izvođenje na resursno ograničenom sklopovlju. Odabir optimalnog rješenja mora se temeljiti na kompromisima (engl. trade-offs) relevantnima za specifičnu domenu rada. Rješenje se za svaku prigodu bira na temelju analize zahtjeva. U Tablici 2.1. prikazana je paralelna usporedba najčešće korištenih posrednika poruka, zajedno sa najčešćim primjenama, prednostima i nedostacima [16] - [18].

Tablica 2.1. Taksonomska usporedba i operativne značajke posrednika poruka

	NATS	Kafka	Redis	RabbitMQ
Najčešća primjena	Obrada tokova niske latencije i edge integracija	Masivni centralizirani cjevovodi analitike	Brzi obradni međuspremnicu u memoriji	Kompleksno usmjeravanje poslovne logike
Složenost	Izuzetno niska (jedna Go binarna datoteka)	Visoka (zahtijeva KRaft ili ZooKeeper topologiju)	Niska do umjerena	Umjerena (Erlang OTP okruženje)
Latencija	< 1ms	Umjerena 10 - 50 ms	< 1ms	Niska 2 - 10ms
Model pohrane	Izborni trajni dnevnik poruka (JetStream)	Obvezni distribuirani dnevnik poruka na disku	Primarno u memoriji (in- memory) uz snapshote	Prolazni redovi poruka s izbornom perzistencijom
Prikladnost za rubne čvorove	Izvrсна (minimalan sklopovski otisak, < 50MB RAM)	Loše (visoki resursni zahtjevi, JVM okruženje)	Umjerena (ograničeno kapacitetom RAM memorije)	Loše (visoka procesorska zahtjevnost)

Ograničenje	Zahtijeva JetStream za semantiku pouzdanosti	Visok operativni i infrastrukturni trošak	Rizik od gubitka podataka pri nestanku napajanja	Pad performansi pri velikom nakupljanju poruka
--------------------	---	---	---	--

Iz tablice 2.1. vidljivo je da sustavi poput Apache Kafke unose preveliku razinu operativne složenosti i zahtijevaju robusne resurse, što ih čini neprikladnima za hibridne modele s izvođenjem na rubu mreže (*edge computing*). Redis postiže niske latencije, ali oslanjanje isključivo na radnu memoriju stvara visoke troškove skaliranja i unosi sigurnosne rizike u slučaju gubitka napajanja. NATS platforma nudi optimalan inženjerski kompromis: pruža iznimno visoke performanse i sub-milisekundne latencije uz minimalan memorijski otisak, dok istodobno kroz JetStream proširenje omogućuje selektivnu aktivaciju trajne pohrane, što izravno opravdava odabir ove tehnologije kao komunikacijske osnove razvijenog prototipa.

2.3. Dvosmjerna komunikacija pomoću WebSocketeta i Socket.io protokola

Kao što različite poslužiteljske komponente distribuiranog sustava komuniciraju na temelju događaja, slično je moguće ostvariti i u komunikaciji poslužitelja s korisničkim sučeljem. U brojnim primjenama, klasični pristup zahtjev-odgovor, u kojem korisnik na zahtjev dobiva podatke s poslužitelja, nije dovoljan. Ako aplikacija zahtijeva stvarnovremenski dotok podataka te njihov prikaz korisniku, najčešće nije praktično da korisnička aplikacija periodički provjerava status slanjem zahtjeva i dohvaćanjem novih podataka. Ovime se, u slučaju niske razine prometa podataka, troše računalni resursi u vidu bespotrebnih zahtjeva, a u slučaju brzog ili nepredvidivog dotoka podataka, pojavljuje se rizik kasnog dobivanja određenog podatka. Zato su osmišljeni protokoli koji rješavaju ovaj problem te postavljaju klijent i poslužitelj u gotovo ravnopravan položaj, uspostavom dvosmjerne i stalne veze. U ovom poglavlju opisuju se protokol WebSocket, koji je najčešće korišteni takav protokol. Također je opisana i apstraktnija varijanta Socket.io, s dodatnim mogućnostima koje izlaze iz okvira izvornog WebSocket protokola.

2.3.1. WebSocket

U klijent – poslužitelj arhitekturama postoje različiti načini komunikacije među sudionicima. Jedan od uobičajenih načina ostvarivanja komunikacije između klijenta i poslužitelja je HTTP protokol [19]. HTTP (*Hyper Text Transfer Protocol*) osnovni je protokol korišten za komunikaciju klijenta i poslužitelja u web tehnologijama. Međutim, on nije idealan za sve vrste primjena. Naime, HTTP protokol temelji se na slanju zahtjeva od strane klijenta. Klijent šalje zahtjev poslužitelju, a poslužitelj odgovara slanjem odgovora koji sadrži tražene podatke ili izvršavanjem tražene akcije. Veza ostvarena tijekom komunikacije HTTP protokolom je privremena. Ona se ponovno uspostavlja prilikom svakog novog zahtjeva, nakon čega se prekida. Ovo može povećati latenciju i usporiti prijenos podataka, što je, iako za većinu slučajeva ne predstavlja značajan problem, ipak značajan faktor u sustavima u kojima se velika količina podataka prenosi u stvarnom vremenu (engl. *real time*). Osim toga, svaka HTTP poruka sadrži i zaglavlje (engl. *header*), što povećava dodatne troškove računalnih resursa (engl. *overhead*). Ovo je posljedica činjenice da je HTTP protokol koji ne čuva stanje (engl. *stateless*). Zato se u zaglavlju svakog pojedinog zahtjeva moraju navesti sve informacije relevantne poslužitelju. Možda najočitija mana HTTP protokola je jednosmjernost komunikacije. Naime, taj protokol dizajniran je na način da klijent šalje zahtjeve poslužitelju. Poslužitelj isključivo odgovara klijentu na zahtjeve, a ne može slati klijentu podatke svojom voljom bez zahtjeva. Potreba za dvosmjernom komunikacijom koja koristi stalnu vezu, u kojoj su klijent i poslužitelj ravnopravni sudionici, s niskim dodatnim troškovima i latencijom, prvenstveno se pojavljuje u sustavima za rad u stvarnom vremenu (engl. *real-time systems*). Rješenje tog problema nudi protokol *WebSocket*.

Protokol *WebSocket* protokol je koji nudi rješenje spomenutih nedostataka HTTP-a. Radi se o protokolu koji uspostavlja stalnu i dvosmjernu vezu između klijenta i poslužitelja. Protokol se temelji na TCP (*Transmission Control Protocol*) protokolu. *WebSocket* protokol je standardiziran po IETF (*Internet Engineering Task Force*) standardizacijskom tijelu, a njegov službeni, standardizacijski naziv je RFC 6455 [20]. Komunikacija započinje uobičajenim HTTP protokolom. Klijent šalje poslužitelju HTTP GET zahtjev, koji sadrži zaglavlje Upgrade, kod kojeg se navodi vrijednost websocket. Time klijent signalizira poslužitelju da želi nadograditi

trenutnu HTTP komunikaciju prelaskom na *WebSocket* protokol. Osim tog zaglavlja, zahtjev sadrži još nekoliko zaglavlja, od kojih je važno zaglavlje *Sec-WebSocket-Key*. Ovom zaglavljem klijent kao vrijednost pridodaje nasumično generiran niz znakova koji on također pohranjuje lokalno. Slanjem ovog zahtjeva započinje proces poznat kao rukovanje (engl. *handshake*) [21]. Po primitku zahtjeva, poslužitelj koristi primljenu vrijednost zaglavlja *Sec-WebSocket-Key* kako bi konstruirao i izračunao novu vrijednost pod nazivom *Sec-WebSocket-Accept*. Dobivenu vrijednost, poslužitelj šalje klijentu u HTTP odgovoru u sklopu zaglavlja *Sec-WebSocket-Accept*. Odgovor koji poslužitelj šalje sadrži HTTP kod *101 – Switching Protocols*, s obzirom da, od tog trenutka nadalje, komunikacije se ostvaruje korištenjem *WebSocket* protokola. Klijent, na temelju vrijednosti *Sec-WebSocket-Key*, koju je i lokalno pohranio, provodi isti postupak izračuna te svoj dobiveni rezultat uspoređuje s rezultatom dobivenim od poslužitelja. Ako se te vrijednosti podudaraju, *WebSocket* komunikacija se uspostavlja. Opisani proces rukovanja nema ulogu autentifikacije niti autorizacije niti osigurava privatnost, tajnost, integritet ili bilo koji drugi uvjet sigurnosti računalnih sustava. To je očito zbog činjenice da je magični string korišten pri izračunavanju vrijednosti *Sec-WebSocket-Accept* zaglavlja javno poznat. Rukovanje ima ulogu pravilnog uspostavljanja *WebSocket* konekcije, a mehanizam izračunavanja i provjere *Sec-WebSocket-Accept* vrijednosti osigurava zaštitu od problema uzrokovanih proxyjima i njihovom priručnom memorijom (engl. *cache*). Svrha tog mehanizma je, također, i osiguravanje da poslužitelj “razumije” *WebSocket* protokol (jer u slučaju da ne razumije, neće imati pristup magičnom stringu).

Danas se *WebSocket* najčešće koristi u sustavima za rad u stvarnom vremenu, chat aplikacijama, platformama za internet stvari (engl. *Internet of Things*), društvenim mrežama i online igrama. Podržavaju ga svi moderni web preglednici, što ga čini uobičajenim i čestim izborom u razvoju programske potpore. Osim osnovne verzije protokola, *ws*, koji koristi nezaštićenu vezu, postoji i protokol *wss*, koji, na opisanu funkcionalnost dodaje i šifriranje korištenjem TLS (engl. *Transport Layer Security*) protokola [22]. Ovo je analogno razlici između HTTP i HTTPS protokola.

Iako je *WebSocket* već sam po sebi moćan protokol, on pruža samo osnovne komunikacijske funkcionalnosti, dok mogućnosti poput dodavanja klijenata u „sobe“, kreiranja

proizvoljnih događaja proizvoljnog imena i sl. nisu izvorno implementirane. Te i mnoge druge funkcionalnosti dostupne su u sklopu Socket.io biblioteke opisane u sljedećem poglavlju.

Na vrhu WebSocket protokola koristi se knjižnica Socket.io. Dok je čisti WebSocket strogo komunikacijski protokol, Socket.io predstavlja apstrakcijski sloj koji klijentima i poslužiteljima donosi napredne mehanizme upravljanja vezom.

2.3.2. Socket.io

Socket.io je biblioteka otvorenog koda koja je temeljena na WebSocket protokolu te izgrađena nad nižerazinskim (engl. *lower level*) Engine.io protokolom [23]. Ona apstrahira taj protokol te uvodi dodatne funkcionalnosti. Socket.io implementacije dostupne su u različitim programskim jezicima, kao što su JavaScript, Python, Java, Go i Rust.

Jedna od osnovnih prednosti Socket.io protokola nad čistim WebSocket protokolom je mogućnost zamjene (engl. *fallback*) načina komunikacije HTTP dugim ispitivanjem (engl. *HTTP long-polling*) u slučaju nemogućnosti korištenja WebSoketa. Naime, iako Socket.io u pozadini prvenstveno koristi WebSocket komunikaciju kad god je to moguće, u slučaju nemogućnosti uspostavljanja WebSocket veze, veza će biti uspostavljena korištenjem HTTP protokola. Do ovoga može doći ako poslužitelj ne podržava WebSocket protokol, iako je to danas rijetkost, ili u slučaju pogrešaka koje onemogućavaju povezivanje WebSocketom. Ovime se osigurava alternativni način povezivanja te omogućava rad s praktički svakim modernim poslužiteljem. Još jedna važna prednost Socket.io protokola nad originalnim WebSocketom je i mogućnost pohrane klijentskih podataka u međuspremnik (engl. *buffering*) te njihovo naknadno slanje u slučaju privremenog gubitka veze. Nedostatak ove mogućnosti mana je WebSoketa te je u svakom iole ozbiljnijem računalnom sustavu, u slučaju korištenja čistog WebSoketa, potrebno implementirati ovu funkcionalnost. Socket.io nudi navedenu funkcionalnost kao svoj sastavni dio i uklanja potrebu za ručnom implementacijom, čime se smanjuju vremenski troškovi rada i smanjuje mogućnost pogrešaka. Jedna od glavnih karakteristika Socket.io biblioteke u slučaju gubitka veze je i automatsko ponovno povezivanje. Socket.io detektira gubitak veze korištenjem tzv. otkucaja srca (engl. *heartbeat*), koji podrazumijevaju periodičku provjeru valjanosti veze. Ako se veza izgubi, Socket.io će je

automatski pokušati ponovno uspostaviti. Još jedna karakteristika Socket.io sustava jest mogućnost definiranja vlastitih događaja te njihovo korištenje u kodu. Nasuprot tome, WebSocket ima nekoliko unaprijed definiranih događaja kojima se implementira sva funkcionalnost. Iako je to često sasvim dovoljno, mogućnost definiranja vlastitih proizvoljnih događaja uvelike povećava fleksibilnost i eleganciju koda pisanog pomoću Socket.io biblioteke. Ovo je posebno korisno kod razvoja sustava koji koriste arhitekturu temeljenu na događajima (engl. *event-based*). Za svaki proizvoljno definirani događaj, također se može definirati i slušatelj, baš kao i za događaje u WebSocketu. U klasičnom WebSocketu, poruke su uvijek namijenjene jednom, točno određenom klijentu. U slučaju potrebe slanja iste poruke više različitim klijenata, potrebno je poslati više poruka, što se obično postiže iteriranjem po listi željenih klijenata. Socket.io nudi rješenje u obliku mogućnosti kreiranja soba, u koje se razvrstavaju klijenti. Poruka se, zatim, može poslati određenoj sobi, pri čemu će poruku primiti svi članovi te sobe. Osim ovoga, moguća je opcija slanja poruke svim povezanim klijentima (engl. *broadcast*).

Ograničenje Socket.io biblioteke je nekompatibilnost s tradicionalnim WebSocket implementacijama. S obzirom na to da Socket.io koristi vlastiti protokol, koji na WebSocket pakete postavlja dodatne metapodatke, Socket.io klijent i čisti WebSocket server nisu kompatibilni te ne mogu ostvariti vezu. Vrijedi i obrnuto. WebSocket klijent ne može ostvariti vezu s Socket.io poslužiteljem.

2.4. Tehnološki stog za implementaciju prototipa

Za implementaciju prototipa u sklopu rada, korišten je skup tehnologija i alata namijenjen programskom ostvarenju različitih dijelova i slojeva aplikacije. Za implementaciju poslužitelja, korišteno je Node.js okruženje, zajedno s Express paketom. Simulatori uređaja ostvareni su koristeći programski jezik Python. Trajna pohrana podataka odvija se pomoću PostgreSQL baze podataka. Ovo poglavlje opisuje navedene tehnologije te njihovu primjenu u kontekstu izrađenog prototipa.

2.4.1. Node.js i Express

Node.js je programsko okruženje namijenjeno implementaciji poslužiteljske strane aplikacija [24]. Njegova glavna uloga i motivacija je omogućavanje korištenja programskog jezika JavaScript kao općenamjenskog programskog jezika, izvan web preglednika. [25] Najčešće se koristi prilikom izrade web aplikacija, iako je prikladan za implementaciju poslužiteljske strane bilo kakve namjene, neovisno o prirodi korisničkog sučelja. Funkcionalnosti za rad s Node.js okruženjem preuzimaju se koristeći *npm* upravljač paketa [26]. Ovo omogućuje preuzimanje minimalnog skupa funkcionalnosti, bez potrebe za prenapuhavanjem (engl. *bloating*).

Express predstavlja jedan od najkorištenijih *npm* paketa u Node.js ekosustavu [27]. To je radni okvir koji apstrahira mnoge funkcionalnosti izrade poslužitelja i omogućava bržu implementaciju funkcionalnosti poput HTTP krajnjih točaka, središnje komponente (engl. *middleware*) i sl. Nudi metode korištene za primanje i obradu HTTP zahtjeva i prijenos informacija HTTP protokolom. Radi se o laganom i minimalističnom radnom okviru koji pruža slobodu i fleksibilnost prilikom dizajna aplikacije i pisanja programskog koda.

2.4.2. Python simulatori izvora događaja

Python, kao visokorazinski programski jezik, pokazuje se osobito prikladan za implementaciju imitatora različitih komponenata programskih sustava. Česta je primjena Pythona za simulaciju izvora događaja. Naime, iako je primarna namjena ovakvih sustava obrada događaja i podataka pristiglih s fizičkih uređaja, kao što su različiti senzori ili mjerni instrumenti, prilikom razvoja aplikacije, nužno je moći testirati funkcionalnost programskih komponenti bez posjedovanja i instalacije fizičkih. Python simulatori događaja su skripte koje se pokreću u proizvoljnom, najčešće velikom, broju instanci te generiraju podatke koji simuliraju format i vrstu podataka pristiglih sa stvarnih uređaja. Simulatori se povezuju na poslužitelj te, putem odabranog protokola, šalju generirane podatke. Mogućnost kreiranja proizvoljnog broja instanci simulatora uređaja važna je jer omogućuje ispitivanje ponašanja

sustava i u uvjetima velikog opterećenja te procjenu potrebe za dodatnim skaliranjem i nadogradnjom postojećeg sustava.

2.4.3. PostgreSQL za trajnu pohranu

PostgreSQL baza podataka je mehanizam trajne pohrane podataka temeljen na relacijskoj shemi podatkovnog modela [28]. Radi se o najčešćoj vrsti trajne pohrane u domeni razvoja korisničkih aplikacija. PostgreSQL koristi SQL upitni jezik za pristup, dodavanje i manipulaciju pohranjenim podacima. Upute nad bazom izvršava PostgreSQL-ov sustav za upravljanje bazom podataka (engl. *Database management system*), koji implementira složene mehanizme optimizacije upita, kontrole pristupa, zaključavanja te osiguravanja ACID (*Atomicity, consistency, isolation, durability*) svojstava. PostgreSQL baze podataka temelje se na podatkovnom relacijskom modelu u kojem su atributi podatka predstavljeni stupcima tablice, a pojedini podaci retcima. Ovakva organizacija podataka omogućuje kompatibilnost s ostalim vrstama reprezentacije podataka u programskim sustavima, kao što su JSON objekti ili objekti razreda u programskim jezicima koji podržavaju objektno orijentirano programiranje. Iz ovoga proizlaze različite naprednije mogućnosti manipulacije bazama podataka, kao što su objektno-relacijsko mapiranje (engl. *object-relational mapping, ORM*). PostgreSQL. Neke od naprednijih funkcionalnosti podržanih u PostgreSQL ekosustavu su pretraživanje teksta, geoprostorne baze podataka (PostGIS), rad s podacima u prozoru i CTE (*Common table expressions*).

2.5. Sažetak odabira tehnologija u odnosu na zahtjeve rada

Tehnološki stog opisan u prethodnim potpoglavljima odabran je za izradu prototipa jer podržava sve potrebne funkcionalnosti i omogućuje ispunjenje zahtjeva na jednostavan i logičan način. Node.js i Express odabrani su za implementaciju poslužiteljske strane aplikacije jer pružaju potrebnu fleksibilnost i omogućuju implementaciju svih potrebnih mogućnosti, bez prenapuhavanja koje bi uzrokovala upotreba alata poput Spring Boota. PostgreSQL baza podataka odabrana je zbog konzistentog podatkovnog modela korištenog u implementaciji, jednostavnosti korištenja te činjenice da je riječ o stabilnoj i zreloj tehnologiji te industrijskom standardu. Socket.io i WebSocket komunikacija industrijski je standard za ostvarivanje dvosmjerne komunikacije u kojoj poslužitelj mora moći klijentu slati poruke i obavijesti. Za

potrebe implementacije, apstrakcija koju uvodi Socket.io ne predstavlja problem, a dodatne mogućnosti koje Socket.io nudi čine ga adekvatnim odabirom. Odabir programskog jezika Python za simulaciju uređaja proizlazi iz činjenice da implementacija i tražena funkcionalnost navedenih uređaja ne zahtijeva nižerazinsku manipulaciju pa jednostavnost Python koda i postojanje svih potrebnih funkcionalnosti u sklopu biblioteka nadmašuju potencijalne nedostatke. Kao posrednik poruka između poslužiteljskih komponenti koristi se NATS, zbog izrazito niske latencije potrebne za sustave koji obrađuju stvarnovremenske podatke. Uz to, prednost je dana NATS-u, u usporedbi s Apache Kafkom i ostalim složenijim posrednicima zbog relativno jednostavnog skupa potrebnih funkcionalnosti, koje ne opravdavaju uvođenje znatno težeg i složenijeg alata.

3. Problem

U ovom poglavlju opisani su funkcionalni i nefunkcionalni zahtjevi izrađenog sustava. Za svaki zahtjev opisani su i načini ispitivanja ispunjenosti te kriteriji na temelju kojih se određuje uspješnost ispunjenja svakog pojedinog zahtjeva. Funkcionalni zahtjevi su prikazani u tablici 3.1.

Tablica 3.1. Prikaz funkcionalnih zahtjeva, načina provjere te kriterija prihvatanja za izrađeni sustav

ID zahtjeva	Tekst zahtjeva	Provjera	Kriteriji prihvatanja
F1	Poslužitelj mora omogućiti periodičko i automatsko primanje podataka s uređaja.	Pokrenuti sustav zajedno s proizvoljnim brojem uređaja te putem ispisanih poruka utvrditi uspješnost slanja podataka.	Uređaj generira podatke periodički, poslužitelj ih prima i u konzoli je vidljiv odgovarajući ispis.
F2	Sustav mora omogućiti trajnu pohranu svih generiranih podataka u bazu podataka.	Pokrenuti sustav zajedno s proizvoljnim brojem uređaja na jednu minutu. Nakon toga provjeriti	Sadržaj baze podataka u potpunosti odgovara generiranim podacima.

		sadržaj baze podataka i usporediti ga s generiranim podacima.	
F3	Sustav mora omogućiti prikaz svih generiranih podataka na korisničkom sučelju.	Pokrenuti sustav zajedno s proizvoljnim brojem uređaja na jednu minutu. Nakon toga provjeriti ispis na korisničkom sučelju i usporediti ga s generiranim podacima.	Sadržaj prikazan na korisničkom sučelju u potpunosti odgovara generiranim podacima.
F4	Sustav mora omogućiti prikaz trenutno povezanih uređaja na korisničkom sučelju.	Pokrenuti sustav zajedno s proizvoljnim brojem uređaja. Provjeriti prikaz uređaja na korisničkom sučelju. Dodavati i uklanjati uređaje te pratiti stanje prikaza na korisničkom sučelju.	Prikaz povezanih uređaja na korisničkom sučelju u potpunosti odgovara povezanim uređajima te ispravno reagira na dodavanje i uklanjanje uređaja.
F5	Sustav mora omogućiti korisnicima zadavanje naredbi uređajima za trenutno slanje podataka.	Pokrenuti sustav zajedno s barem dva uređaja. Pokušati poslati po nekoliko naredbi svakom od njih. Pratiti ispis na korisničkom sučelju.	Prikaz podataka na korisničkom sučelju reflektira uspješno generiranje podataka na korisnički zahtjev.
F6	Sustav mora omogućiti razlikovanje pohranjenih i nepohranjenih podataka na temelju prikaza na korisničkom sučelju.	Pokrenuti sustav bez komponente za pohranu i obradu podataka te pratiti prikaz podataka na korisničkom sučelju. Zatim pokrenuti komponentu za pohranu i obradu podataka te pratiti prikaz na korisničkom sučelju.	Razlika između pohranjenih i nepohranjenih podataka jasno vidljiva u ispisu na korisničkom sučelju prikazom odgovarajuće ikone uz podatak.
F7	Sustav mora omogućiti prikaz povijesnih podataka.	Pokrenuti sustav bez uređaja te pratiti prikaz na korisničkom sučelju.	Na zaslonu se prikazuju podaci koji u potpunosti

		Usporediti ga sa sadržajem baze podataka.	odgovaraju sadržaju baze podataka.
F8	Sustav mora omogućiti brisanje svih podataka.	Pokrenuti sustav s proizvoljnim brojem uređaja na jednu minutu. Provjeriti sadržaj baze podataka te prikaz na korisničkom sučelju. Prekinuti rad svih uređaja te inicirati brisanje podataka. Ponovno provjeriti sadržaj baze podataka te prikaz na korisničkom sučelju.	Nakon brisanja podataka, baza podataka ne sadrži podatke, a korisničko sučelje ne prikazuje niti jedan zapis podatka.

Tablica 3.2. opisuje nefunkcionalne zahtjeve sustava.

Tablica 3.2. Prikaz nefunkcionalnih zahtjeva, načina provjere te kriterija prihvatanja za izrađeni sustav

ID zahtjeva	Tekst zahtjeva	Provjera	Kriteriji prihvatanja
N1	Sustav mora omogućiti povezivanje proizvoljnog broja uređaja.	Pokrenuti sustav s jednim uređajem. Zatim s 10 pa sa 100 uređaja. Pratiti ponašanje sustava.	Sustav omogućuje povezivanje uređaja bez izazivanja pogreške.
N2	Sustav mora ostvariti komunikaciju između poslužiteljskih komponenti neizravno, korištenjem posrednika poruka.	Pokrenuti sustav te pratiti njegov rad. Pratiti sadržaj koji posrednik poruka logira. Prekinuti rad posrednika poruka. Pratiti rad sustava.	Sustav korektno radi za vrijeme rada posrednika poruka, a logirane poruke odražavaju stvarno stanje komunikacije komponenti. Po prestanku rada posrednika poruka, sustav prestaje s radom.

N3	Sustav mora ostvariti nisku latenciju između generiranja podatka te njegove pohrane u bazu podataka pri normalnim uvjetima rada.	Pokrenuti sustav s manjim brojem uređaja, uspoređivati vremenske oznake nastale pri generiranju podatka te pri upisu u bazu podataka.	Latencija manja od 50 ms za barem 95% zapisa u bazi podataka.
N4	Komponente sustava moraju posjedovati jasnu podjelu odgovornosti.	Uvidom u programski kod utvrditi odgovornost svake pojedine komponente.	Odgovornosti su jasno razdvojene i podijeljene po komponentama. Niti jedna poslužitejska komponenta ne smije imati više od dvije osnovne funkcionalnosti.
N5	Sustav mora omogućiti visoku propusnost za podatke.	Pokretati sustav sa sve većim brojem uređaja koji šalju podatak periodički svake sekunde. Pratiti ponašanje sustava te zapise u bazi podataka i korisničkom sučelju.	Uz 50 pokrenutih uređaja, sadržaj baze podataka I sadržaj prikazan na korisničkom sučelju moraju odgovarati stvarnim poslanim podacima po broju poruka. Latencija mora biti niža od 100 ms za barem 95% poruka. Ne smije doći do rušenja poslužitelja ili prestanka rada sustava na bilo koji način.

4. Metodologija i oblikovanje rješenja

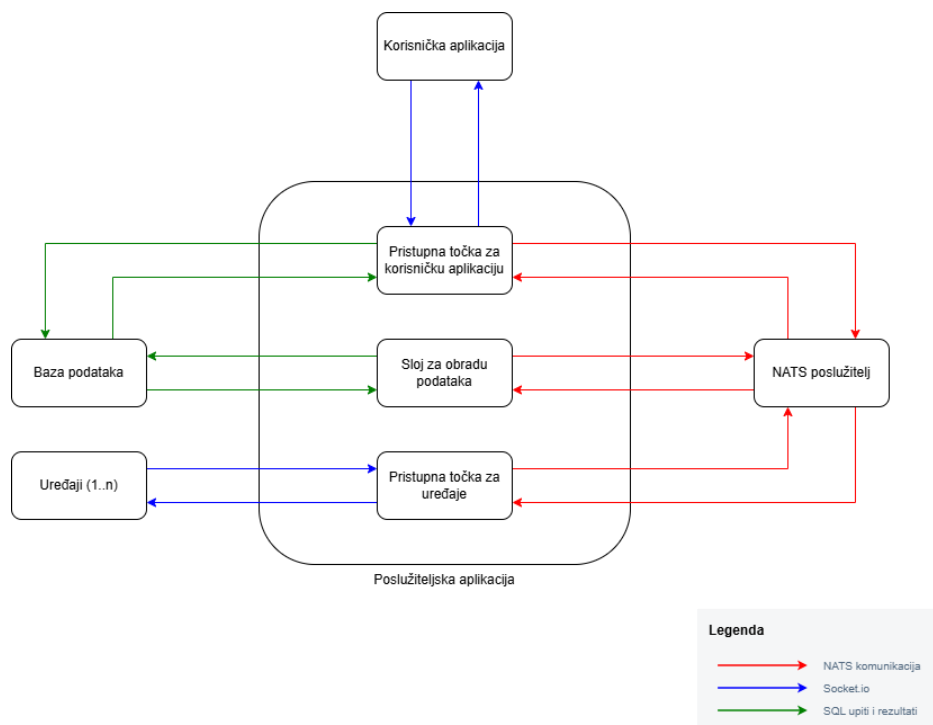
U ovom se poglavlju definira konceptualno oblikovanje i arhitektura proširivog informacijskog sustava za obradu događaja u stvarnom vremenu. Sustav je strukturiran kao distribuirani asinkroni cjevovod poruka s hibridnim modelom izvođenja poslovne logike na rubu mreže (edge sloj) i u oblaku (cloud sloj). Arhitekturni dizajn eliminira izravne prostorne i vremenske ovisnosti između komponenti te definira jasne mehanizme za obradu tokova podataka, trajnu pohranu i evaluaciju pravila u stvarnom vremenu.

4.1. Raspodijeljena komponentna arhitektura s topologijom komunikacijskih predmeta i modelom pohrane

U sklopu rada, razvijena je aplikacija koja simulira rad sustava za generiranje, obradu, pohranu i prikaz podataka. Aplikacija simulira rad sustava u kojem uređaji generiraju podatke, šalju ih na poslužitelj, nakon čega se podaci prikazuju korisniku putem korisničkog sučelja te pohranjuju u bazu podataka. Podaci se na uređajima generiraju periodički ili na zahtjev korisnika, u željenom trenutku. Sustav se sastoji od četiri glavne komponente: uređaja, poslužiteljske strane aplikacije (engl. *backend*), korisničkog sučelja (engl. *frontend*) te baze podataka. Sama poslužiteljska strana aplikacije sastoji se od tri podkomponente: pristupne točke za uređaje (engl. *device gateway*), sloja za obradu podataka (engl. *data processor*) te pristupne točke za korisničko sučelje (engl. *frontend gateway*). Pristupna točka za uređaje je dio poslužiteljske strane koji komunicira s uređajima, prima podatke od njih te im prosljeđuje korisničke naredbe. Sloj za obradu podataka je dio poslužiteljske strane zadužen za obradu i pohranu podataka u bazu podataka. Pristupna točka za korisničko sučelje šalje podatke korisničkom sučelju te prima korisničke zahtjeve koje prosljeđuje ostalim dijelovima poslužitelja ili bazi podataka. Navedena arhitektura ostvaruje podjelu odgovornosti među različitim dijelovima poslužitelja te modularnost, što rezultira čistim i jasnijim kodom te olakšanim uvođenjem promjena u sustav.

Uređaji su implementirani kao skripte u programskom jeziku Python. Ovime se omogućuje provjera funkcionalnosti sustava bez potrebe za nabavljanjem i instalacijom fizičkih uređaja. Poslužiteljska strana aplikacije implementirana je koristeći Node.js okruženje uz dodatak Express biblioteke. Klijentska strana aplikacije implementirana je kao klasična korisnička web aplikacija, korištenjem označnih jezika HTML i CSS te jezika JavaScript. Za trajnu pohranu podataka koristi se PostgreSQL baza podataka. U skladu sa spomenutom podjelom odgovornosti i modularnošću sustava, komponente sustava međusobno komuniciraju različitim protokolima i vrstama poruka. Naime, tri komponente poslužiteljske strane ne komuniciraju direktno, već posredstvom NATS poslužitelja, koristeći objave i pretplate na NATS predmete, kao i funkcionalnost zatjeva i odgovora. Sam poslužitelj s uređajima i s korisničkim sučeljem komunicira korištenjem protokola Socket.io, što je važno kako bi se ostvarila dvosmjerna komunikacija te omogućio rad u stvarnom vremenu, gdje komponente

reagiraju na događaje pristigle s poslužitelja, bez potrebe za provjerama ili slanjem zahtjeva. Za komunikaciju poslužitelja s bazom podataka, koriste se SQL upiti. Navedena arhitektura na visokoj razini, prikazana je dijagramom na slici 4.1.

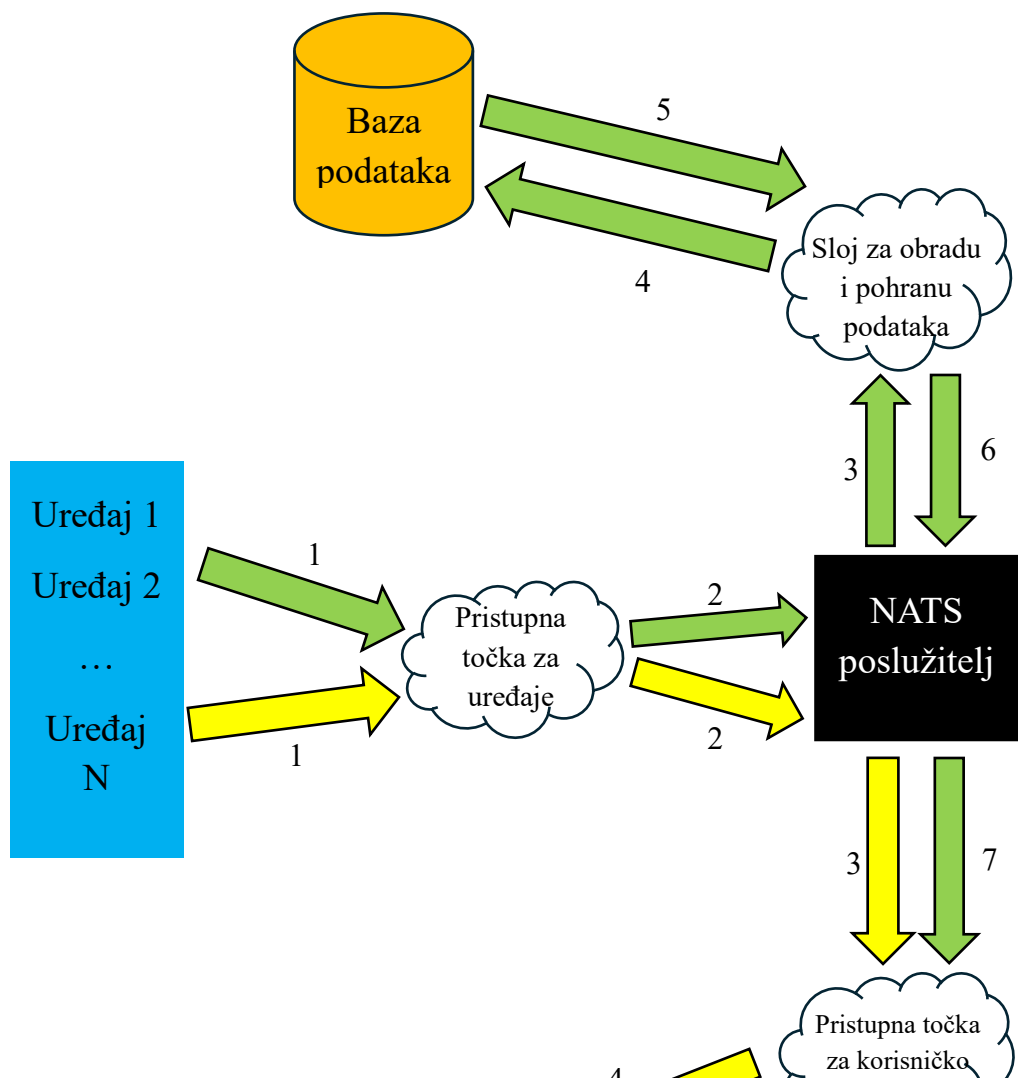


Slika 4.1. Dijagram komponenti sustava s prikazom komunikacijskih puteva

4.2. Model asinkronih tokova događaja od simulatora uređaja i životni ciklus poruke

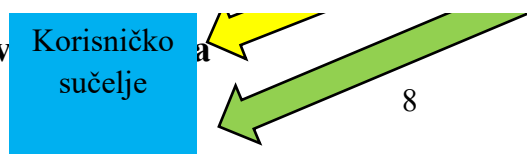
Tok podataka unutar cjevovoda strogo je asinkron kako bi se osiguralo minimalno blokiranje pošiljatelja u uvjima visoke frekvencije prometa. Životni ciklus poruke definiran je kroz sekvencijalni niz koraka koji eliminiraju sinkrono čekanje diskovnog upisa pri mrežnom prihvatu. Tok poruke od izvora do krajnjeg korisničkog sučelja prikazan je na slici 4.2.

Različitim bojama strelica naglašena su dva različita puta podataka u sustavu: žute strelice označavaju put poruka bez pohrane u bazu podataka, dok zelene označavaju put koji uključuje pohranu u bazu prije proslijeđivanja poruke korisniku. Redni brojevi uz strelice prikazuju redoslijed koraka puta poruke kroz sustav.



Slika 4.2. Prikaz puta poruka od uređaja do korisničkog sučelja

4.3. Matrica sljediv



U ovom poglavlju prikazana je matrica sljedivosti zahtjeva (engl. *requirements traceability matrix*). Ona povezuje prethodno definirane zahtjeve s konkretnim dijelovima

razvijenog programskog sustava, kako bi način ispunjenja svakog pojedinog zahtjeva bio u potpunosti jasan. Matrica sljedivosti zahtjeva prikazana je u Tablici 4.1.

Tablica 4.1. Matrica sljedivosti zahtjeva razvijenog programskog rješenja

Zahtjev (ID)	Tekst zahtjeva	Implementacijski element	Provjera
F1	Poslužitelj mora omogućiti periodičko i automatsko primanje podataka s uređaja.	Simulatori uređaja, pristupna točka za uređaje, Socket.io događaj <code>device-data</code>	Pokrenuti sustav zajedno s proizvoljnim brojem uređaja te putem ispisanih poruka utvrditi uspješnost slanja podataka.
F2	Sustav mora omogućiti trajnu pohranu svih generiranih podataka u bazu podataka.	Baza podataka, sloj za obradu i pohranu podataka	Pokrenuti sustav zajedno s proizvoljnim brojem uređaja na jednu minutu. Nakon toga provjeriti sadržaj baze podataka i usporediti ga s generiranim podacima.
F3	Sustav mora omogućiti prikaz svih generiranih podataka na korisničkom sučelju.	Korisničko sučelje, pristupna točka za korisničko sučelje, Socket.io događaj <code>new-data</code>	Pokrenuti sustav zajedno s proizvoljnim brojem uređaja na jednu minutu. Nakon toga provjeriti ispis na korisničkom sučelju i usporediti ga s generiranim podacima.
F4	Sustav mora omogućiti prikaz trenutno povezanih uređaja na korisničkom sučelju.	Korisničko sučelje, pristupna točka za korisničko sučelje, Socket.io događaj <code>devices-update</code> , NATS predmeti <code>devices.status</code> i <code>devices.list</code>	Pokrenuti sustav zajedno s proizvoljnim brojem uređaja. Provjeriti prikaz uređaja na korisničkom sučelju. Dodavati i uklanjati uređaje te pratiti stanje prikaza na korisničkom sučelju.
F5	Sustav mora omogućiti korisnicima zadavanje naredbi uređajima za trenutno slanje podataka.	Korisničko sučelje, Socket.io događaji <code>request-data</code> i <code>send-data-now</code> , NATS predmeti	Pokrenuti sustav zajedno s barem dva uređaja. Pokušati poslati po nekoliko naredbi svakom od njih. Pratiti ispis na korisničkom sučelju.

		devices.commands.[U REDAJ_ID]	
F6	Sustav mora omogućiti razlikovanje pohranjenih i nepohranjenih podataka na temelju prikaza na korisničkom sučelju.	Korisničko sučelje, Socket.io događaji new-data i database-data.	Pokrenuti sustav bez komponente za pohranu i obradu podataka te pratiti prikaz podataka na korisničkom sučelju. Zatim pokrenuti komponentu za pohranu i obradu podataka te pratiti prikaz na korisničkom sučelju.
F7	Sustav mora omogućiti prikaz povijesnih podataka.	Korisničko sučelje, Pristupna točka za korisničko sučelje, HTTP krajnja točka /podaci s metodom GET	Pokrenuti sustav bez uređaja te pratiti prikaz na korisničkom sučelju. Usporediti ga sa sadržajem baze podataka.
F8	Sustav mora omogućiti brisanje svih podataka.	Korisničko sučelje, Pristupna točka za korisničko sučelje, HTTP krajnja točka /podaci s metodom DELETE	Pokrenuti sustav s proizvoljnim brojem uređaja na jednu minutu. Provjeriti sadržaj baze podataka te prikaz na korisničkom sučelju. Prekinuti rad svih uređaja te inicirati brisanje podataka. Ponovno provjeriti sadržaj baze podataka te prikaz na korisničkom sučelju.

5. Implementacija

U ovom poglavlju opisuje se implementacija programskog prototipa za prihvata, obradu, pohranu i prikaz događaja u blizu stvarnom vremenu. Poglavlje se nadovezuje na prethodno definirane zahtjeve i arhitekturu sustava te prikazuje kako su pojedine odgovornosti raspodijeljene među simulatorima uređaja, poslužiteljskim komponentama, korisničkim sučeljem i bazom podataka.

Razvijeni prototip sastoji se od više odvojenih komponenti koje zajedno ostvaruju tok događaja od simuliranog uređaja do korisničkog prikaza i trajne pohrane. Uređaji su

implementirani kao Python simulatori, dok je poslužiteljska strana izvedena u okruženju Node.js. Međusobna komunikacija poslužiteljskih komponenti ostvarena je pomoću NATS posrednika poruka, a komunikacija s uređajima i korisničkim sučeljem pomoću Socket.io protokola. Podjela odgovornosti među komponentama uvedena je kako bi se izbjeglo izravno spajanje svih dijelova sustava. Pristupna točka za uređaje prihvata događaje i naredbe prema uređajima, sloj obrade preuzima zapisivanje događaja u bazu podataka, a pristupna točka za korisničko sučelje prilagođava događaje web aplikaciji. Takva podjela olakšava kasnije ispitivanje pojedinih dijelova toka obrade i smanjuje međusobnu ovisnost komponenti. U tablici 5.1. prikazane su komponente sustava zajedno s korištenim tehnologijama i skupom odgovornosti.

Tablica 5.1. Komponente sustava, korištene tehnologije i odgovornosti

Komponenta	Tehnologija	Odgovornost
Simulator izvora događaja	Python, Socket.io klijent	Generiranje periodičkih događaja i odgovaranje na zahtjev za trenutačnim slanjem podatka.
Pristupna točka za uređaje	Node.js, Socket.io, NATS	Registracija uređaja, prihvati podataka s uređaja i proslijeđivanje naredbi odabranom uređaju.
Sloj obrade i pohrane	Node.js, NATS, PostgreSQL	Primanje događaja s NATS predmeta, pohrana u bazu i objava potvrđeno pohranjenog zapisa.
Pristupna točka za korisničko sučelje	Node.js, Express, Socket.io, NATS	Prosljeđivanje događaja korisničkom sučelju, dohvat povijesnih podataka i proslijeđivanje korisničkih naredbi.
Korisničko sučelje	HTML, CSS, JavaScript, Socket.io klijent	Prikaz aktualnih i povijesnih podataka, prikaz povezanih

		uređaja i slanje korisničkih zahtjeva.
Baza podataka	PostgreSQL	Trajna pohrana događaja i vremenskih oznaka potrebnih za provjeru obrade.

5.1. Simulator uređaja

Simulator uređaja je komponenta implementirana kao Python skripta koja oponaša ponašanje fizičkog uređaja. Pokretanjem više instanci skripte moguće je simulirati istovremeni rad većeg broja uređaja, pri čemu svakoj instanci korisnik dodjeljuje ID. ID se određuje pri pokretanju skripte kao argument naredbenog retka. Navedeni ID služi isključivo za prikaz uređaja na korisničkom sučelju aplikacije te nije nužno jedinstveni identifikator uređaja. Nakon pokretanja skripte i unosa ID-ja, kreira se Socket.io klijentski objekt, nakon čega se uređaj spaja na poslužitelj i to na adresu <http://localhost:3001> jer je to upravo adresa pristupne točke za uređaje. Spajanje se odvija koristeći naredbu: `sio.connect("http://localhost:3001")`. Odmah nakon uspješnog uspostavljanja Socket.io veze s poslužiteljem, dolazi do pojave Socket.io događaja `connect`, koji se obrađuje u istoimenoj funkciji slušatelju događaja. U toj funkciji, ispisuje se poruka o uspješnosti povezivanja te se, putem upravo uspostavljene Socket.io veze, poslužitelju šalje događaj `register`, čime se poslužitelju omogućuje da zabilježi povezani uređaj te da se isti prikaže na korisničkom sučelju. Ulaz u prikazani dio koda je događaj `connect`, a izlaz je događaj `register` poslan poslužitelju zajedno s identifikatorom uređaja. Ovaj dio koda povezan je sa zahtjevom F4, jer se emitiranjem događaja `register` omogućuje ažuriranje popisa svih povezanih uređaja.

```
@sio.event
def connect():
    print("Povezivanje uspješno")
    sio.emit("register", DEVICE_ID)
```

Po povezivanju, simulator ulazi u stanje čekanja na Socket.io događaje koji predstavljaju naredbu za slanjem podataka poslanu uređaju. Periodičko slanje izvedeno je u zasebnoj pozadinskoj dretvi kako bi simulator mogao istodobno generirati događaje i čekati naredbe pristigle preko Socket.io veze. Novi podatak generira se i šalje svakih 5 sekundi, što je ostvareno u funkciji `periodic()`. Ovaj dio koda povezan je sa zahtjevom F1 jer omogućuje periodičko i automatsko generiranje i slanje podataka s uređaja prema poslužitelju.

```
def periodic():
    while True:
        time.sleep(5)
        send_data()

threading.Thread(target = periodic, daemon = True).start()
```

Izvršavanjem posljednje prikazane linije koda pokreće se demonska dretva (engl. *daemon*), što znači da se njeno izvršavanje automatski zaustavlja u trenutku završetka glavne dretve. Svaki generirani događaj sadrži identifikator uređaja, jedinstveni identifikator podatka, vremensku oznaku te sam podatak. `uredjaj_id` predstavlja ID koji je korisnik unio. `podatak` je sam generirani nasumični broj, a `vrijeme` predstavlja vremensku oznaku (engl. *timestamp*) generiranja podatka. `podatak_id` je identifikator svakog pojedinog generiranog podatka, . Jedinstveni identifikator podatka kasnije se koristi za povezivanje neposredno prikazane kopije događaja s potvrđenom kopijom pohranjenom u bazu podataka. Navedeno se postiže u funkciji

send_data. Ovaj dio koda povezan je sa zahtjevom F1, jer se u njemu generiraju i šalju podaci na poslužitelj.

```
def send_data():
    podaci = {
        "uredjaj_id": DEVICE_ID,
        "podatak_id": str(uuid.uuid4()),
        "podatak": random.randint(1, 100),
        "vrijeme": datetime.datetime.now(datetime.timezone.utc).isoformat()
    }
    sio.emit("device-data", podaci)
```

Naredbe za trenutnim slanjem podataka uređaji primaju putem Socket.io događaja send-data-now te funkcije slušatelja navedenog događaja, u kojoj se ispisuje poruka o zahtjevu te podaci šalju korištenjem već opisane funkcije. Funkcija slušatelj navedenog događaja prikazan je u nastavku. Ulaz u prikazani dio programa je događaj send-data-now pristigao s poslužitelja, a izlaz je poziv funkcije send_data, kojim se inicira generiranje i slanje podataka, kako je ranije opisano. Ovaj dio koda povezan je sa zahtjevom F5 jer se radi o obradi događaja nastalog korisničkim zahtjevom za trenutnim slanjem podataka.

```
@sio.on("send-data-now")
def on_command():
    print("Zahtjev za podacima stigao")
    send_data()
```

Uređaj radi navedene aktivnosti sve do gašenja programa, čime se Socket.io veza prema poslužitelju automatski prekida te se ka poslužitelju šalje Socket.io događaj disconnect. Navedeno je sastavni dio načina na koji implementacija Socket.io klijenta radi. Više instanci simulatora može pokrenuti istodobno, simulator čini osnovu za ispitivanje ponašanja sustava pri različitim opterećenjima. Za potrebe eksperimenata preporučljivo je interval generiranja

učiniti konfigurabilnim parametrom, jer se time omogućuje kontrolirano povećavanje opterećenja bez izmjene programskog koda.

5.2. Poslužiteljska aplikacija

Poslužiteljska strana prototipa podijeljena je na tri glavne komponente: pristupnu točku za uređaje, sloj obrade i pohrane događaja te pristupnu točku za korisničko sučelje. Komponente ne pozivaju izravno funkcije drugih komponenti, nego podatke i naredbe razmjenjuju preko NATS predmeta. Time se ostvaruje labavo povezivanje i omogućuje da svaka komponenta obavlja jasno određenu odgovornost.

U implementiranom prototipu za razmjenu poruka među poslužiteljskim komponentama koristi se NATS Core. Takav odabir dovoljan je za ostvarenje niske latencije i labavog povezivanja komponenti, dok se trajnost događaja ostvaruje na razini PostgreSQL baze podataka. JetStream nije korišten u osnovnoj verziji prototipa jer bi uveo dodatne mehanizme trajnih tokova i potvrda primitka koji nisu nužni za provjeru osnovnog toka obrade, ali predstavljaju mogući smjer nadogradnje u sustavu s jačim zahtjevima pouzdanosti.

5.2.1. Pristupna točka za uređaje

Pristupna točka za uređaje dio je poslužitelja zadužen za komunikaciju s uređajima te posredstvo između njih i ostalih dijelova poslužitelja. Ova komponenta prima podatke od uređaja te ih prosljeđuje ostalim dijelovima sustava, a istovremeno prima zahtjeve od ostalih dijelova sustava te odgovara na njih ili ih prosljeđuje uređajima. Najprije se na adresi *<http://localhost:3001>* kreira HTTP poslužitelj, nad kojim se, zatim, kreira Socket.io poslužitelj. Poslužitelj je konfiguriran koristeći CORS (*Cross-origin request sharing*) postavke koje omogućuju povezivanje s pristupnom točkom za korisničko sučelje na adresi *<http://localhost:3000>*. Objekt `io` nastao kao povratna vrijednost kreiranja poslužitelja, objekt je nad kojim se izvršavaju metode vezane za poslužitelj, tj. za sve povezane uređaje. Sama funkcionalnost pristupne točke za uređaje ostvarena je u obliku slušatelja Socket.io događaja

(za komunikaciju s uređajima) te metoda kojima se ostvaruje pretplata/objavljivanje na odgovarajući NATS predmet (za komunikaciju s ostalim dijelovima poslužitelja). Svaki registrirani uređaj dodaje se u Socket.io sobu čiji naziv odgovara identifikatoru uređaja. Time pristupna točka za uređaje može naredbu poslati točno određenom uređaju bez ručnog pretraživanja svih aktivnih Socket.io veza. Ovaj pristup pojednostavljuje usmjeravanje naredbi i izravno se povezuje s NATS predmetima oblika `devices.commands.[ID_UREĐAJA]`.

Početnu točku izvođenja programa predstavlja funkcija `start()`, koja se poziva odmah nakon kreiranja poslužitelja te se u njoj ostvaruje povezivanje na NATS poslužitelj, pretplate na odgovarajuće NATS predmete te definiraju potrebni slušači događaja. Najprije se ostvaruju pretplate na predmete `"devices.commands.>"` te `"devices.list"` i pokreću se odgovarajuće metode `handlerKomandi` i `handlerListeUredjaja`. Nakon toga slijedi definicija slušatelja događaja `connection`, koji pristiže kad god se novi uređaj poveže na poslužitelj. Argument te funkcije slušatelja je objekt `socket`, koji predstavlja trenutnog klijenta. Međutim, konkretna funkcionalnost rukovanja uređajima i konekcijama ostvarena je u ugniježđenim slušateljima za događaje `register`, `device-data` i `disconnect`. Kad se primi događaj `register`, ID uređaja pohranjuje se na već spomenuti objekt `socket`, a zatim se registrirani uređaj dodaje u odgovarajuću sobu definiranu vlastitim ID-jem. Kako bi se održavao popis trenutno povezanih uređaja te kako bi se povezao ID uređaja koji je korisnik postavio s ID-jem dodijeljenim od strane Socket.io protokola, te dvije vrrijednosti se, kao par ključ-vrijednost, dodaju u mapu `uredaji`. Registracija uređaja završava objavom liste uređaja dobivene iz navedene mape na NATS predmet `devices.status`, što omogućuje ostalim dijelovima sustava dobivanje podataka o trenutno povezanim uređajima. Navedeno je vidljivo u isječku koda 5.1. Ulaz u ovaj isječak koda predstavlja događaj `connection` pristigao na poslužitelj te objekt `socket`, koji predstavlja povezani uređaj, dok su događaj `register` i ID povezanog uređaja ulaz u ugniježđeni dio koda. Izlaz iz ugniježdene funkcije slušatelja predstavlja objava ažurirane liste uređaja na NATS predmet `devices.status`. Ovaj isječak povezan je sa zahtjevima F4, N1 i N2 jer omogućuje praćenje trenutno povezanih uređaja putem liste, a opisani mehanizam registracije podržava proizvoljan broj povezanih uređaja. Također, korištenje NATS objave predstavlja usklađenost s zahtjevom prema kojem je obavezna indirektna komunikacija između komponenti sustava, korištenjem posrednika poruka.

Isječak 5.1. Funkcija slušatelj događaja connection na pristupnoj točki za uređaje

```
io.on("connection", (socket) => {
  console.log("Uredaj spojen:", socket.id);

  socket.on("register", (deviceId) => {
    socket.deviceId = String(deviceId);
    socket.join(socket.deviceId);
    uredaji.set(socket.deviceId, socket.id);
    console.log("Uredaj registriran", socket.deviceId);

    nc.publish("devices.status", sc.encode(JSON.stringify({
      event: "connected",
      uredaji: Array.from(uredaji.keys())
    })));
  });

});

// HANDLERI device-data i disconnect
});
```

Kad pristupna točka za uređaje primi podatke od uređaja, to se događa putem Socket.io događaja `device-data`. Ovaj se događaj obrađuje u funkciji slušatelju, koja ispisuje poruku o primanju podataka te objavljuje dobivene podatke na NATS predmet koji odgovara uređaju pošiljatelju. Naime, predmeti na koje se podaci objavljuju imenovani su tako da posljednji dio naziva predmeta odvojen točkom odgovara ID-ju uređaja koji je poslao podatak. Šablona koju nazivi tih predmeta koriste je `devices.data.[ID_UREĐAJA]`. Navedeno je prikazano u Isječku 5.2. Ulaz u prikazani isječak je događaj `device-data` te `socket` objekt, koji predstavlja povezani uređaj. Izlaz iz isječka predstavlja objava podataka s uređaja na NATS predmet `devices.data.[UREĐAJ_ID]`. Ovaj isječak povezan je sa zahtjevima F1 i N2 jer se radi o

funkcionalnosti ključnoj za primanje i nastavak obrade podataka pristiglih s uređaja, a vidljiva je i objava na NATS predmet, što ukazuje na indirektnu komunikaciju među poslužiteljskim komponentama.

Isječak 5.2. Funkcija slušatelj za događaj device-data

```
io.on("connection", (socket) => {  
  //... HANDLER za register  
  
  socket.on("device-data", (data) => {  
    console.log("Podaci primljeni s uređaja", data);  
    nc.publish(`devices.data.${data.uredjaj_id}`,  
      sc.encode(JSON.stringify(data)));  
  });  
  //HANDLER za disconnect  
  
});
```

Prilikom odspajanja uređaja s poslužitelja, automatski se šalje Socket.io događaj `disconnect`. Po njegovom primanju, iz mape uređaji briše se par ključ-vrijednost koji odgovara netom odspojenom uređaju. Nakon toga se ažurirana informacija o trenutno spojenim uređajima čini dostupnom objavljivanjem popisa uređaja na NATS predmet `devices.status`, baš kao što je slučaj i pri spajanju novog uređaja.

Ranije spomenute pretplate na NATS predmete `"devices.commands.>"` te `"devices.list"` služe za komunikaciju s ostalim dijelovima poslužitelja, konkretno s pristupnom točkom za korisničko sučelje, koja je objašnjena detaljnije u narednim poglavljima. Pretplatom na predmet `devices.commands.>` ostvaruje se primanje naredbi za trenutno slanje podataka koje treba proslijediti odgovarajućem uređaju. Svaki uređaj pridružen je jednom NATS predmetu naziva `devices.commands.[ID_UREĐAJA]`. ID željenog uređaja pristiže kao

sadržaj poruke. Nakon dekodiranja sadržaja, naredba se proslijeđuje željenom uređaju slanjem Socket.io događaja `send-data-now` sobi u kojoj se uređaj nalazi, kao što je prikazano Isječkom 5.3. Ulaz u ovaj isječak su objekt `cmdSub`, rezultat pretplate po kojemu se može iterirati i koji sadrži poruke pristigle na temu `devices.commands.>` te objekt `io`, koji predstavlja sve veze prema povezanim uređajima. Ovaj isječak povezan je sa zahtjevima F5 i N2 jer predstavlja funkcionalnost ključnu za proslijeđivanje korisničkih naredbi koje omogućuju generiranje podataka na zahtjev, a čitanje pristiglih naredbi putem pretplate predstavlja neizravnu komunikaciju između komponenata poslužitelja.

Isječak 5.3. Funkcija `handlerKomandi` na pristupnoj točki za uređaje

```
async function handlerKomandi(cmdSub, io) {
  for await (const poruka of cmdSub) {
    const { deviceId } = JSON.parse(sc.decode(poruka.data));
    console.log("Saljem naredbu uređaju ", deviceId);
    io.to(deviceId).emit("send-data-now");
  }
}
```

Pristupna točka za uređaje definira i funkciju kojom se obrađuju podaci pristigli s NATS predmeta `devices.list`. Isječak 5.4. prikazuje tu funkciju. Ovaj isječak povezan je sa zahtjevima F4 i N2 jer, na zahtjev druge komponente, šalje aktualnu listu povezanih uređaja, što omogućuje njihov pravilan prikaz na korisničkom sučelju. Također, mehanizam pretplate te objave odgovora na NATS zahtjev odražavaju indirektnu komunikaciju između komponenata.

Isječak 5.4. Funkcija `handlerListeUredaja` na pristupnoj točki za uređaje

```
async function handlerListeUredaja(listSub, io) {
  for await (const poruka of listSub) {
    poruka.respond(sc.encode(JSON.stringify({uredaji:
      Array.from(uredaji.keys())})));
  }
}
```

}

Ovom komponentom ostvaruju se zahtjevi povezani s registracijom uređaja, prihvatom događaja s više izvora i prosljeđivanjem korisničkih naredbi prema odabranom uređaju. Budući da se ulazni događaji odmah objavljuju na NATS predmet, pristupna točka za uređaje ne pohranjuje podatke, čime se zadržava jasna podjela odgovornosti među komponentama.

5.2.2. Sloj obrade i pohrane podataka

Sloj za obradu i pohranu podataka komponenta je poslužitelja čija je osnovna odgovornost trajna pohrana događaja u PostgreSQL bazu podataka i objava potvrđeno pohranjenog zapisa. Ova komponenta sluša na adresi <http://localhost:3002> te je jedina koja ne koristi Socket.io komunikaciju. Naime, komponenta nema izravnu komunikaciju s uređajima ni korisničkim sučeljem, nego podatke prima isključivo preko NATS pretplate.

Sloj obrade i pohrane događaja prima podatke koje pristupna točka za uređaje objavljuje na NATS predmete oblika `devices.data.>`. Zato rad ovog sloja započinje povezivanjem s NATS poslužiteljem te pretplatom na NATS predmet `devices.data.>`, s kojeg pristižu navedeni podaci. Nakon pretplate, na poruke se čeka u funkciji `handlerPodataka`, koja pohranjuje dobivene podatke u bazu, a odmah zatim prosljeđuje netom umetnuti redak pristupnoj točki za korisničko sučelje, kako bi se na korisničkom sučelju mogla prikazati potvrda o pohrani podataka. Pohrana u bazu podataka ostvaruje se jednostavnim SQL upitom, bez korištenja ORM-a (*object-relational mapping*) ili posebnih radnih okvira za rukovanje bazama podataka. Dobivanje netom umetnutog retka ostvaruje se ključnom riječi `RETURNING`, koja se postavlja na kraj upita. Taj se umetnuti redak zatim prosljeđuje pristupnoj točki za korisničko sučelje objavom na NATS predmet `database.data`, namijenjen podacima koji ne dolaze direktno s uređaja, već su trajno pohranjeni. Primanje podataka, njihova pohrana u bazu te prosljeđivanje na NATS predmet prikazani su Isječkom 5.5. Ulaz u ovaj dio programa je objekt pretplate `dataSub` pomoću kojeg se dobivaju podaci te objekt NATS veze, `nc`. Izlaz iz navedenog isječka je objava netom upisanih podataka u bazu na NATS predmet `database.data`. Ovaj isječak

povezan je sa zahtjevima F2 i N2 jer pokazuje kako je izvedena trajna pohrana podataka u bazu, a korištenje NATS posrednika ukazuje na indirektnu komunikaciju komponenata.

Isječak 5.5. Funkcija handlerPodataka na sloju za obradu i pohranu podataka

```
async function handlerPodataka(dataSub, nc) {
  for await (const poruka of dataSub) {
    const podaci = JSON.parse(sc.decode(poruka.data));
    console.log("Pohranjujem podatke: ", podaci);

    const res = await pool.query("INSERT INTO podaci (podatak_id, uredjaj_id,
podatak, vrijeme) VALUES ($1, $2, $3, $4) RETURNING *", [podaci.podatak_id,
String(podaci.uredjaj_id), podaci.podatak, podaci.vrijeme]);

    const noviRedak = res.rows[0];
    nc.publish("database.data", sc.encode(JSON.stringify(noviRedak)));
    console.log("Šaljem pohranjene podatke", noviRedak);

  }
}
```

Samo povezivanje s bazom podataka ostvareno je korištenjem funkcionalnosti koje pruža *npm* paket *pg*, namijenjen radu s PostgreSQL bazama podataka. Koristi se povezivanje putem bazena (engl. *pool*), što podrazumijeva skup SQL veza koje se mogu koristiti, osloboditi te ponovno upotrijebiti i predstavlja učinkovitiji način korištenja baze podataka s gledišta računalnih resursa. Vjerodajnice za povezivanje s bazom podataka nalaze se u *.env* datoteci, čime se izbjegava njihovo unošenje direktno u programski kod. Veza s bazom ostvarena je pomoću paketa *pg* i bazena veza, pri čemu se parametri povezivanja učitavaju iz konfiguracijske datoteke okruženja.

Sloj za obradu podataka ujedno je i jedina komponenta sustava s mogućnošću pisanja u bazu podataka. Ostale komponente, kao što je pristupna točka za korisničko sučelje, mogu po

potrebi čitati već pohranjene podatke, no sama pohrana zadatak je isključivo sloja za obradu podataka. Razlog tome je podjela odgovornosti i buduće nadogradnje, koje će uključivati validaciju i sanitizaciju podataka prije upisivanja u bazu. Navedenim pristupom, logika kojom se to postiže bit će lokalizirana na jednom mjestu, umjesto da bude ponovno implementirana više puta u različitim komponentama.

5.2.3. Pristupna točka za korisničko sučelje

Pristupna točka za korisničko sučelje dio je poslužitelja zadužen za direktnu komunikaciju s korisničkim sučeljem aplikacije (engl. *frontend*). Komunikacija se ostvaruje Socket.io protokolom te HTTP rest pozivima, ovisno o traženoj funkcionalnosti. Ova pristupna točka pokreće se na adresi <http://localhost:3000>. Komponenta se pretplaćuje na NATS predmet `devices.data.>`, namijenjen primanju sirovih podataka s uređaja koje pristupna točka za uređaje prosljeđuje. Pripadna funkcija koja obrađuje navedenu pretplatu je `handlerPodataka`, koja dobivene podatke prosljeđuje neizmjenjene korisničkom sučelju koristeći Socket.io događaj `new-data`, kao što prikazuje isječak 5.6. Ulaz u ovaj dio koda je objekt `dataSub`, koji predstavlja objekt nastao pretplatom na NATS predmet `devices.data.>` i objekt `io`, koji predstavlja Socket.io poslužitelj, tj. vezu sa svim povezanim komponentama. Izlaz iz navedenog isječka je slanje podataka korisničkom sučelju u obliku Socket.io događaja `new-data`. Ovaj kod povezan je sa zahtjevima F3 i N2 jer prikazuje ključni korak za prikaz podataka s uređaja na korisničkom sučelju, a korištenje NATS pretplate za primanje podataka s druge poslužiteljske komponente pokazuje indirektnu komunikaciju.

Isječak 5.6. Funkcija `handlerPodataka` na pristupnoj točki za korisničko sučelje

```
async function handlerPodataka(dataSub, io) {  
  for await (const poruka of dataSub) {  
    const podaci = JSON.parse(sc.decode(poruka.data));  
    console.log("Saljem podatke na frontend: ", podaci);  
    io.emit("new-data", podaci);  
  }  
}
```

```

    }
}

```

Komponenta se također pretplaćuje na predmet `devices.status`, s kojeg prima aktualnu listu povezanih uređaja nakon svakog novog spajanja i odspajanja. Navedenu listu, u funkciji `handlerStatusa`, proslijeđuje korisničkom sučelju, što omogućuje prikaz odgovarajućih gumba korisniku te dinamičko ažuriranje istih. Slanje liste korisničkom sučelju ostvareno je Socket.io događajem `devices-update`. Navedena je funkcionalnost prikazana Isječkom 5.7. Ulaz u navedeni isječak su objekt pretplate `statusSub` te Socket.io objekt `io`. Izlaz predstavlja slanje popisa trenutno povezanih uređaja korisničkom sučelju koristeći Socket.io događaj `devices-update`. Ovaj isječak povezan je sa zahtjevima F4 i N2 jer omogućuje proslijeđivanje liste trenutno povezanih uređaja korisničkom sučelju, što je nužno za dinamički prikaz istih. A korištenje NATS pretplate za dobivanje podataka od druge komponente demonstrira arhitekturu u kojoj komponente indirektno komuniciraju putem posrednika poruka.

Isječak 5.7. Funkcija `handlerStatusa` na pristupnoj točki za korisničko sučelje

```

async function handlerStatusa(statusSub, io) {
  for await (const poruka of statusSub) {
    const { uredaji } = JSON.parse(sc.decode(poruka.data));
    console.log("Azuriranje uredaja: ", uredaji);
    io.emit("devices-update", uredaji);
  }
}

```

Posljednja NATS pretplata korištena u ovoj komponenti je na predmet `database.data` te pripadna funkcija `handlerPodatakaIzBaze`. Ovom pretplatom se primaju netom pohranjeni podaci od sloja za obradu podataka te se proslijeđuju korisničkom sučelju putem Socket.io događaja `database_data`. Može se primijetiti da, iako se isti podaci već šalju korisničkom sučelju u funkciji `handlerPodataka`, ti podaci su oni direktno pristigli s pristupne točke za uređaje, bez pohrane u bazu. Podaci poslani u ovoj funkciji predstavljaju potvrđeno spremljenu

kopiju podataka. Ovime se postiže povećanje pouzdanosti, potvrdom korisniku da su pristigli podaci pohranjeni, a istovremeno se zadržava prednost direktnog slanja podataka bez pohrane, u vidu smanjene latencije. Navedeno je prikazano u Isječku 5.8. Ulaz u ovaj dio koda su objekt NATS pretplate `databaseSub` i Socket.io objekt `io`. Izlaz iz navedenog isječka je slanje podataka iz baze putem Socket.io događaja `database_data`. Ovaj isječak povezan je sa zahtjevima F6 i N2, s obzirom na to da slanje podataka iz baze odvojenim kanalom olakšava njihovo razlikovanje od podataka pristiglih direktno, bez pohrane. Također, korištenje NATS pretplate za primanje podataka s druge komponente pokazuje indirektnu komunikaciju komponenti.

Isječak 5.8. Funkcija `handlerPodatakaIzBaze` na pristupnoj točki za korisničko sučelje

```
async function handlerPodatakaIzBaze(databaseSub, io) {  
  for await (const poruka of databaseSub) {  
    const podaci = JSON.parse(sc.decode(poruka.data));  
    console.log("Saljem podatke iz baze na frontend: ", podaci);  
    io.emit("database_data", podaci);  
  }  
}
```

Socket.io protokol ova komponenta koristi isključivo za komunikaciju s korisničkim sučeljem. Pri pokretanju korisničkog sučelja, ono se povezuje na pristupnu točku te pritom generira Socket.io događaj `connection`. Obrada ovog događaja podrazumijeva dobavljanje liste trenutno povezanih uređaja. Naime, iako se prilikom svakog novog povezivanja ili odspajanja uređaja objavljuje NATS poruka, zbog neperzistentnosti NATS komunikacije, informacije pristigle prije početka rada ove komponente neće biti primljene. Kako bi se osigurao uvid u situaciju u trenutku pokretanja korisničkog sučelja, neovisno o prethodnim događajima, šalje se NATS zahtjev na predmet `devices.list`, nakon čega se očekuje odgovor. Odgovor šalje pristupna točka za uređaje, kao što je prethodno opisano, u obliku trenutne liste spojenih uređaja.

U sklopu funkcije slušatelja koja obrađuje događaj povezivanja, ugniježđeno se definira funkcija koja obrađuje Socket.io događaj request-data. To je događaj koji s korisničkog sučelja pristiže kad korisnik, pritiskom na gumb, zatraži trenutno generiranje i slanje podataka s određenog uređaja, izvan uobičajenog 5-sekundnog intervala. Pristupna točka za korisničko sučelje preuzima sadržaj poslan uz ovaj događaj, koji podrazumijeva ID željenog uređaja te prosljeđuje naredbu pristupnoj točki za uređaje putem već spomenutog NATS predmeta oblika `devices.commands.[ID_UREĐAJA]`. Kad uređaj pošalje podatke, oni prolaze isti put kao i periodički generirani podaci te se, kao što je ranije opisano, primaju iz dva različita smjera i prosljeđuju korisničkom sučelju. Programski kod funkcije slušatelja za događaj connection, zajedno sa svim ugniježđenim slušateljima, u cijelosti je prikazan u isječku 5.9. Ulaz u navedeni isječak su Socket.io događaj connection te Socket.io objekt socket, koji predstavlja vezu prema povezanom korisničkom sučelju. Izlaz iz prikazanog dijela koda je izazivanje nekog od prikazanih događaja te njegovo slanje putem Socket.io veze. Ovaj isječak povezan je sa zahtjevom F5, jer sadrži poslužiteljsku funkcionalnost koja omogućuje korisnicima zahtijevanje trenutnog slanja podataka.

```
io.on("connection", async (socket) => {
  console.log("Frontend povezan:", socket.id);

  try {
    const reply = await nc.request(
      "devices.list",
      sc.encode(""),
      { timeout: 1000 }
    );
    const { uredaji } = JSON.parse(sc.decode(reply.data));
    socket.emit("devices-update", uredaji);
  } catch (err) {
    console.log("Nemoguće dohvatiti listu uredaja:", err.message);
  }
});
```

```

socket.on("request-data", (deviceId) => {
    console.log("Frontend traži podatke od uređaja:", deviceId);
    nc.publish(
        `devices.commands.${deviceId}`,
        sc.encode(JSON.stringify({ deviceId })))
    );
});

socket.on("disconnect", () => {
    console.log("Frontend odspojen:", socket.id);
});
});

```

Isječak 5.9. Obrada događaja connection te ugniježđeni slušatelji na pristupnoj točki za korisničko sučelje

Pristupna točka za korisničko sučelje pruža i klasične HTTP krajnje točke (engl. *endpoint*) REST sučelja, koje služe dohvaćanju i brisanju svih podataka iz baze. Dohvaćanje podataka iz baze vrši se odmah nakon učitavanja korisničkog sučelja u svrhu prikaza povijesnih podataka, dok brisanje inicira korisnik pritiskom gumba. Navedene radnje izvode se povezivanjem na bazu podataka na način ekvivalentan opisanom u prethodnom poglavlju te izvršavanjem SQL upita. Opisana funkcionalnost implementirana je na način prikazan isječkom 5.10. Ulaz u ovaj dio koda predstavljaju HTTP GET i DELETE zahtjevi, upućeni krajnjoj točki /podaci, zajedno sa objektima req i res koji su namijenjeni obradi zahtjeva te odgovoru na njega. Izlaz predstavljaju pozivi metode res.json, koji vraćaju podatke iz baze, odnosno potvrđuju uspješnost brisanja podataka, ovisno o korištenoj HTTP metodi. Ovaj isječak povezan je sa zahtjevima F7 i F8 jer prikazuje kod koji omogućuje dohvat povijesnih podataka iz baze te brisanje istih podataka.

Isječak 5.10. HTTP krajnje točke za dohvat i brisanje podataka na pristupnoj točki za korisničko sučelje

```
app.get("/podaci", async (req, res) => {
  const result = await pool.query("SELECT * FROM podaci ORDER BY vrijeme");
  console.log((await result).rowCount);
  res.json(result.rows);
})

app.delete("/podaci", async (req, res) => {
  const result = await pool.query("DELETE FROM podaci");
  res.json({success: true});
})
```

5.3. Korisničko sučelje

Korisničko sučelje (engl. *frontend*) dio je aplikacije koji se izvodi na klijentskom računalu korisnika, u web pregledniku. Implementirano je kao web aplikacija korištenjem HTML-a, CSS-a i JS-a. Korisničko sučelje se učitava s adrese <http://localhost:3000> te ostvaruje vezu s pristupnom točkom za korisničko sučelje putem Socket.io protokola za kontinuiranu stvarnovremensku komunikaciju i protokola HTTP za sporadične zahtjeve. Odmah nakon učitavanja korisničkog sučelja, započinje proces inicijalizacije, koji podrazumijeva dohvat povijesnih podataka iz baze te njihov prikaz na zaslonu. Ti se podaci dohvaćaju slanjem HTTP zahtjeva na adresu <http://localhost:3000/podaci>, a rezultat zahtjeva objekt je koji sadrži sve podatke do tada pohranjene u bazi podataka. S obzirom na to da, za vrijeme dohvaćanja povijesnih podataka iz baze, sustav radi te kontinuirano prima nove podatke od uređaja, kako bi se izbjeglo njihovo miješanje s povijesnim podacima, dupliciranje ili izmjena redoslijeda prikaza, podaci koji pristižu za vrijeme dohvata povijesnih podataka pohranjuju se u međuspremnik, ostvaren varijablom buffer. Nakon dohvata povijesnih podataka, međuspremnik se prazni te se njegovi podaci prikazuju na zaslonu, kao i povijesni. Dohvat povijesnih podataka implementiran je u funkciji `fetchData`, prikazanoj Isječkom 5.11. Ulaz u ovaj isječak je poziv funkcije `fetchData()`, a izlaz je uspješan završetak izvođenja ili iznimka

u slučaju neuspjeha. Ovaj isječak povezan je sa zahtjevom F7, jer demonstrira iniciranje dohvata povijesnih podataka pri učitavanju aplikacije.

Isječak 5.11. Dohvat povijesnih podataka funkcijom `fetchData` u korisničkoj aplikaciji

```
async function fetchData() {
  try {
    const res = await fetch('http://localhost:3000/podaci');
    const data = await res.json();
    if (data.length === 0) return;

    const list = document.getElementById('data-list');
    list.innerHTML = '';

    data.slice().forEach(item => {
      prikaziPodatak(item, true);
    });
    stariPodaciUcitani = true;
  } catch (err) {
    console.error("Could not load historical data:", err);
  }
}
```

Funkcija `loadBufferedData`, koja služi za prikaz podataka međuspremnik trivijalna je i sastoji se od iteriranja međuspremnikom te poziva funkcije za prikaz podatka na svakom njegovom elementu, kao što je prikazano u nastavku. Ulaz u ovaj dio koda je poziv funkcije `loadBufferedData`, a izlaz iz njega je poziv funkcije `prikaziPodatak`. Ovaj dio programa povezan je sa zahtjevom F3 jer osigurava pravilan prikaz svih generiranih podataka na korisničkom sučelju.

```
async function loadBufferedData() {
```

```

    for (const item of buffer) {
        prikaziPodatak(item, false);
    }
}

```

Funkcija za prikaz povijesnih podataka iz baze te funkcija za prikaz podataka međuspremnik koriste funkciju `prikaziPodatak`, koju pozivaju nad svakim elementom iterabilnog objekta čije je podatke potrebno prikazati. U toj funkciji nalazi se dijeljena logika koja je zajednička za svaki dio programa zadužen za prikazivanje podataka na zaslonu. Ova funkcija prima dva argumenta. Prvi argument je sam podatak koji je potrebno prikazati, primljen Socket.io protokolom od pristupne točke za korisničko sučelje. Drugi argument specificira prirodu samog podatka. Vrijednost `true` koristi se ako je podatak pristigao iz baze podataka, a `false` indicira da se radi o podatku pristiglom direktno s uređaja, bez kontakta s bazom podataka. Kao što je ukratko objašnjeno u prethodnim poglavljima, pristupna točka za korisničko sučelje, a time i sama korisnička aplikacija, dobivaju dvije kopije svakog novopristiglog podatka. U standardnim uvjetima mreže, kopija koja najprije pristiže je podatak koji s uređaja i pristupne točke za uređaje direktno prelazi na pristupnu točku za korisničko sučelje. Ova verzija podatka pristiže brže, jer ne podliježe latenciji baze podataka te dodatne NATS objave potrebne za prijenos na sloj za obradu podataka. Međutim, u trenutku pristizanja tog podatka na korisničko sučelje, njegova trajna pohranjenost u bazu podataka još nije potvrđena te se, pored prikaza podatka na korisničkom sučelju, prikazuje ikona pješčanog sata, što simbolizira obradu u tijeku i čekanje na konačnu potvrdu. Najčešće nakon kratke latencije, pristiže i verzija podatka koja je pohranjena u bazu podataka. Ona je s pristupne točke za uređaje najprije poslana na sloj za obradu podataka, pohranjena u bazu, a tek tada, kao redak baze, preuzeta i proslijeđena pristupnoj točki za korisničko sučelje te samom korisničkom sučelju. Ovakav podatak trajno je pohranjen u bazu te predstavlja podatak koji je zasigurno pohranjen u sustav. Zato se, nakon pristizanja ove kopije podatka, pored njegovog prikaza na korisničkom sučelju pojavljuje ikona kvačice.

Nakon poziva funkcije, najprije se provjerava je li željeni podatak već prethodno bio prikazan. Naime, `prikazaniIdevi` je skup (engl. *set*) u koji se dodaje ID svakog novoprikazanog podatka te služi za deduplikaciju. S obzirom da se ova funkcija koristi i za ispis

podataka iz baze i za ispis podataka koji pristižu direktno, ID predstavljen vrijednošću atributa `item.podatak_id` je već spomenuti UUID4 identifikator. Korištenje klasične jedinstvene ID vrijednosti nije moguće jer se ona dodjeljuje isključivo podacima pohranjenima u bazu podataka, ali ne i onima pristiglima direktno. Ovime se izbjegava dvostruki ispis istog podatka na korisničkom sučelju. Ako podatak nije već prikazan, locira se HTML element korisničkog sučelja, korištenjem alata za manipulaciju dokumentno-objektnog modela korisničkog sučelja (engl. *DOM, Document object model*) namijenjen prikazu zapisa podataka. Zatim se kreira novi HTML element tipa `<li>`, u koji se, zatim, upisuju pristigli podaci u odgovarajućem formatu. Implementacija ove funkcije prikazana je u Isječku 5.12. Ulaz u prikazani dio programa je poziv funkcije `prikaziPodatak` te njeni argumenti: `podatak` te informacija o tome je li podatak pohranjen u bazi. Izlaz iz ovog isječka je uspješno izvršavanje funkcije ili povratak iz funkcije bez dodavanja novog prikaza podatka. Ovaj isječak povezan je sa zahtjevima F3 i F6 jer sadrži dijeljenu logiku korištenu na nekoliko mjesta namijenjenih prikazu podataka na korisničkom sučelju, a ujedno sadrži i mehanizam razlikovanja pohranjenih i nepohranjenih podataka prikazom na korisničkom sučelju.

Isječak 5.12. Funkcija `prikaziPodatak` u korisničkoj aplikaciji

```
function prikaziPodatak(item, baza = false) {
  if (prikazaniIdevi.has(item.podatak_id)) {
    return;
  } else {
    prikazaniIdevi.add(item.podatak_id);
    const list = document.getElementById('data-list');
    const placeholder = list.querySelector('.no-data');
    if (placeholder) placeholder.remove();

    const li = document.createElement('li');
    li.dataset.podatakId = item.podatak_id;
    if (baza) {
      li.textContent = `✅ Uređaj ${item.uredjaj_id}: ${item.podatak} – ${new
Date(item.vrijeme).toLocaleTimeString()}`;
    }
  }
}
```

```

    } else {
        li.textContent = ` ⌚ Uređaj ${item.uredjaj_id}: ${item.podatak} – ${new
Date(item.vrijeme).toLocaleTimeString()}`;
    }

    list.prepend(li);
}
}

```

U skladu s tim, pozivi funkcije `prikaziPodatak` u prethodno prikazanim odsječcima postaju jasni. Naime, prilikom učitavanja povijesnih podataka, svi učitani podaci već se nalaze u bazi pa se funkcija za prikaz poziva s argumentom `true`. U funkciji za prikaz podataka međuspremnik, funkcija se izvršava s argumentom `false`, jer se međuspremnik puni isključivo podacima direktno pristiglima bez pohrane.

Do sada prikazani kod odnosi se na prikaz povijesnih podataka te podataka međuspremnik pristiglih za vrijeme inicijalizacije. Nakon završetka navedenih aktivnosti, korisnička aplikacija započinje uobičajeni način rada u kojem prima podatke od pristupne točke za korisničko sučelje te ih kontinuirano obrađuje. Podaci se primaju putem Socket.io protokola i to putem dva različita definirana događaja. Događaj `new-data` služi primanju nepohranjenih podataka direktno s uređaja, dok je događaj `database-data` namijenjen primanju pohranjenih događaja iz baze podataka. Ovakvim odvajanjem uklanja se potreba za naknadnim provjerama porijekla pristiglih podataka te dodatnim i složenijim deduplikacijskim mehanizmima. Prilikom obrade događaja `new-data`, najprije se provjerava je li učitavanje povijesnih podataka dovršeno, što se postiže provjerom vrijednosti varijable `stariPodaciUcitani`. Provjera je nužna jer, ako povijesni podaci još nisu učitani, umjesto prikaza, podatak se sprema u međuspremnik te kasnije obrađuje. U uobičajenom scenariju u kojem su povijesni podaci već učitani, podatak se prikazuje pozivom funkcije za prikaz s argumentom `false`, tj. s prikazom pješčanog sata. Dva su moguća scenarija prilikom poziva ove funkcije. Uobičajeni scenarij je da ovaj podatak pristiže prvi, jer kopija podatka iz baze kasni zbog latencije. U tom slučaju, program radi kako je opisano, prikazom pješčanog sata pored podatka. A ako je, zbog neuobičajenih mrežnih uvjeta, npr. usporenja veze između pristupne točke za uređaje i pristupne točke za korisničko

sučelje, u trenutku pristizanja ove kopije podatka, perzistentna kopija iz baze već ranije bila pristigla i bila prikazana, funkcija `prikaziPodatak` neće učiniti ništa, već će, pri prvoj provjeri je li podatak prikazan, završiti s izvođenjem. Implementacija slušatelja događaja `new-data` prikazana je u Isječku 5.13. Ulaz u ovaj dio koda je primanje `Socket.io` događaja `new-data` te objekt `item`, koji sadrži novopristigle podatke. Izlaz iz isječka je ili poziv funkcije `prikaziPodatak` ili dodavanje podatka u međuspremnik. Ovaj isječak povezan je sa zahtjevom F3 jer prikazuje mehanizam primanja podataka, ključan za omogućavanje prikaza svih generiranih podataka na korisničkom sučelju. Indirektno je povezan i sa zahtjevom F6 jer predstavlja logiku korištenu isključivo za prikaz nepohranjenih podataka, dok je prikaz pohranjenih obrađen drugamo.

Isječak 5.13. Slušatelj `Socket.io` događaja `new-data` u korisničkoj aplikaciji

```
socket.on("new-data", (item) => {  
  if (stariPodaciUcitani) {  
    prikaziPodatak(item, false);  
  } else {  
    buffer.push(item);  
  }  
});
```

Slušatelj događaja `database-data` sličan je prethodnom. Nakon pristizanja podatka iz baze, postoje dva scenarija. Prvi scenarij je uobičajeni, u kojem je podatak direktno s uređaja već pristigao. U tom slučaju nije potrebno prikazivati podatak ponovno, nego samo promijeniti znak pješčanog sata u znak kvačice. Drugi scenarij podrazumijeva izmijenjeni redoslijed, u kojem je podatak iz baze prvopristigli. U tom slučaju, funkcija za prikaz podatka se poziva s argumentom `true`. Slušatelj koristi metodu `document.querySelector`, koja filtrira po kriteriju `UUID4` identifikatora podatka, za pronalazak odgovarajućeg zapisa podatka, a rezultat izvršavanja ove metode govori o kojem se scenariju radi, tj. je li podatak s uređaja već pristigao. Ako je odgovarajući element pronađen, zapis već postoji. Ako se ne pronađe traženi element, zapisa još nema i treba ga prikazati. Implementacija funkcije slušatelja događaja `database-`

data prikazana je u Isječku 5.14. Ulaz u ovaj dio koda je pristizanje događaja `database_data` te objekt `item`, koji sadrži pristigle podatke. Izlaz iz ovog dijela koda je poziv funkcije `prikaziPodatak` ili manipulacija sadržaja korisničkog sučelja. Ovaj isječak povezan je sa zahtjevom F6 jer osigurava razlikovanje pohranjenih i nepohranjenih podataka promjenom ikone.

Isječak 5.14. Slušatelj Socket.io događaja `database_data` u korisničkoj aplikaciji

```
socket.on("database_data", (item) => {
  const li = document.querySelector(`[data-podatak-id="${item.podatak_id}"]`);
  if (li) {
    li.textContent = li.textContent.replace("⌚", "✅");
  } else {
    prikaziPodatak(item, true);
  }
})
```

Osim prikaza podataka, sučelje pruža i funkciju za jednostavno brisanje svih podataka iz baze. Ovo se postiže pozivom funkcije `obrisiSvePodatke`. Implementacija te funkcije trivijalna je i sastoji se od API poziva prema adresi <http://localhost:3000/podaci>, s http metodom DELETE. Nakon toga se unutarnji HTML `<ul>` elementa za prikaz podataka briše, kako bi promjena odmah postala vidljiva korisniku. Na kraju se isprazni skup `prikazaniIdevi`.

Korisničko sučelje sve vrijeme prikazuje i trenutno povezane uređaje. Svakom promjenom popisa povezanih uređaja, generira se događaj `devices-update` i šalje na korisničko sučelje. Primanjem ovog događaja prima se i popis ID vrijednosti svih trenutno povezanih uređaja. Po primljenom se objektu iterira, za svaki element kreira se gumb koji se prikazuje na korisničkom sučelju s odgovarajućom ID vrijednosti. Svakom se gumbu, također, pripisuje i funkcija koja se poziva po pritisku tog gumba. Radi se o funkciji `requestDevice(id)`, koja se poziva za onaj ID uređaja na koji se gumb odnosi. Ta funkcija šalje Socket.io događaj `request-data` pristupnoj točki za korisničko sučelje, što korisniku pruža mogućnost zahtijevanja trenutnog slanja podatka od bilo kojeg trenutno povezanog uređaja. Ako niti jedan uređaj nije

spojen, na zaslonu se prikazuje tekst koji o tome izvještava korisnika. Implementacija navedene funkcije slušatelja događaja `devices-update` prikazana je u Isječku 5.15. Ulaz u ovaj isječak predstavlja događaj `devices-update` te argument `devices`, koji predstavlja trenutni popis povezanih uređaja. Izlaz iz ovog dijela koda nastupa uspješnim iteriranjem kroz popis te završavanjem funkcije. Isječak je povezan sa zahtjevima F4 i F5 jer je u njemu implementirana logika prikaza trenutno povezanih uređaja, a u istoj funkciji slušatelju, svakom dodanom gumbu pridružuje se i funkcija koja inicira slanje zahtjeva za trenutnim generiranjem podataka.

Isječak 5.15. Funkcija slušatelj Socket.io događaja `devices-update` u korisničkoj aplikaciji

```
socket.on("devices-update", (devices) => {  
  const container = document.getElementById('device-buttons');  
  container.innerHTML = '';  
  
  if (devices.length === 0) {  
    container.innerHTML = '<span class="no-data">Nema uređaja</span>';  
    return;  
  }  
  
  devices.forEach(id => {  
    const btn = document.createElement('button');  
    btn.textContent = `Zatraži podatke od uređaja ${id}`;  
    btn.onclick = () => requestDevice(id);  
    container.appendChild(btn);  
  });  
});
```

5.4. Podatkovni model i trajna pohrana

Trajna pohrana događaja u sustavu postignuta je primjenom PostgreSQL relacijske baze podataka. Baza podataka sadrži jednu tablicu pod nazivom `podaci`, u koju se pohranjuju zapisi

pristigli sa sloja za obradu podataka. U Tablici 5.2. prikazani su stupci tablice podaci, njihov tip podatka te uloga u sustavu.

Tablica 5.2. Stupci tablice podaci u bazi podataka, njihov tip i uloga

Stupac	Tip	Uloga
id	SERIAL	Tehnički primarni ključ zapisa u bazi.
podatak_id	UUID UNIQUE	Jedinstveni identifikator događaja nastao u simulatoru; koristi se za deduplikaciju i povezivanje kopija događaja.
uredjaj_id	TEXT	Identifikator uređaja koji je proizveo događaj.
podatak	INTEGER	Vrijednost koju je simulator generirao.
vrijeme	TIMESTAMPZ	Vremenska oznaka nastanka događaja na uređaju.
vrijeme_spremanja	TIMESTAMPZ	Vremenska oznaka pohrane događaja u bazu podataka.

Prilikom dodavanja podataka u bazu, kao što je vidljivo u Tablici 5.2., podaci se proširuju dodavanjem novih stupaca/atributa, što je implementirano na razini same baze podataka. Dodaje se jedinstveni identifikator svakog pojedinog podatka, kao cijeli broj (engl. *integer*), što ujedno predstavlja i primarni ključ (engl. *primary key*) svih podataka u bazi. Osim toga, podacima se pridodaje i vremenska oznaka pohrane u bazu, predstavljena stupcem `vrijeme_spremanja`. Ovo se razlikuje od vremenske oznake dodane podacima pri generiranju na uređaju jer određeno vrijeme protekne od generiranja podatka do njegove pohrane u bazu podataka. Ovo omogućuje i analizu latencije sustava te čini vidljivom razliku između vremena samog događaja te vremena pohrane. Veća latencija od uobičajene može biti znak usporavanja mreže ili neke od komponenti sustava. Osim navedenih dodanih stupaca, ostali stupci baze odgovaraju već postojećim atributima podataka generiranih na uređaju. Ti stupci tablice zovu se: `podatak_id` (UUID4 identifikator podatka), `uredjaj_id` (identifikator uređaja koji unosi korisnik), `podatak` (nasumični broj koji je uređaj generirao) i `vrijeme` (vremenska oznaka trenutka generiranja podatka). CREATE SQL upit kojim je tablica kreirana prikazan je u isječku 5.16. Isječak je povezan sa zahtjevom F2 jer se u njemu kreira tablica nužna za trajnu pohranu svih generiranih podataka.

Isječak 5.16. SQL upit kojim je kreirana tablica za pohranu podataka

```
CREATE TABLE podaci(  
    id SERIAL PRIMARY KEY,  
    podatak_id UUID UNIQUE,  
    uredjaj_id TEXT,  
    podatak INTEGER,  
    vrijeme TIMESTAMPTZ,  
    vrijeme_spremanja TIMESTAMPTZ NOT NULL DEFAULT NOW()  
)
```

Baza podataka puštena je u pogon (engl. *deploy*) korištenjem Render usluge s besplatnom opcijom. Iako su, odabirom te opcije, mogućnosti glede skalabilnosti, kapaciteta pohrane i dostupnosti baze niže od standarda industrije, ipak se pokazuju adekvatnima za svrhu testiranja prototipa.

5.5. Pregled načina komunikacije i sudionika

Ovo poglavlje tablično prikazuje elemente komunikacije korištene za ostvarivanje veze između komponenti sustava. Ovo uključuje NATS predmete zajedno s objavljiivačima i pretplatnicima, Socket.io događaje zajedno s pošiljateljima i primateljima te poslužiteljske krajnje točke namijenjene HTTP zahtjevima.

5.5.1. Pregled NATS predmeta i sudionika

U Tablici 5.3. prikazani su svi NATS predmeti korišteni za komunikaciju između komponenti poslužitelja te je dan pregled komponenti pretplaćenih na svaki predmet, kao i komponenti koje objavljuju na iste predmete. Također je dan kratak opis primjene svakog

predmeta. Oznake (REQ) i (RES) u tablici govore da se radi o slanju NATS zahtjeva/odgovora na navedenu temu.

Tablica 5.3. Pregled NATS predmeta zajedno s kontekstom unutar sustava

NATS PREDMET	Objavlivači	Pretplatnici	Sadržaj poruke	Opis
devices.status	Pristupna točka za uređaje	Pristupna točka za korisničko sučelje	{event, uređaji}	Popis trenutno povezanih uređaja
devices.data.[UREĐAJ_ID]	Pristupna točka za uređaje	Sloj za obradu podataka, Pristupna točka za korisničko sučelje	data	Prosljeđivanje podatka dobivenog s uređaja
devices.commands.[UREĐAJ_ID]	Pristupna točka za korisničko sučelje	Pristupna točka za uređaje	deviceId	Prosljeđivanje naredbe upućene uređaju
devices.list	Pristupna točka za korisničko sučelje (REQ), Pristupna točka za uređaje (RES)	Pristupna točka za uređaje	""	Zahtjev za dobivanjem liste trenutno povezanih uređaja
database.data	Sloj za obradu podataka	Pristupna točka za korisničko sučelje	noviRedak	Prosljeđivanje podataka pohranjenih u bazu podataka

5.5.2. Pregled socket.io događaja i sudionika

U Tablici 5.4. prikazani su svi Socket.io događaji koji se koriste za komunikaciju poslužitelja s uređajima i s korisničkim sučeljem te pošiljatelj i primatelj svakog događaja. Dan je i kratak opis značenja svakog događaja u sustavu.

Tablica 5.4. Pregled Socket.io događaja zajedno s kontekstom unutar sustava

SOCKET.IO DOGAĐAJ	Pošiljatelj	Primatelj	Sadržaj poruke	Opis
register	Uređaji	Pristupna točka za uređaje	DEVICE_ID	Registracija uređaja/dodavanje na popis spojenih uređaja
device-data	Uređaji	Pristupna točka za uređaje	podaci	Slanje podataka s uređaja na poslužitelj
connect	Pristupna točka za uređaje (automatski)	Uređaji	-	Događaj generiran pri povezivanju uređaja
send-data-now	Pristupna točka za uređaje	Uređaji	-	Zahtjev uređaju za trenutnim slanjem podataka
connection	Uređaji (automatski), Korisničko sučelje (automatski)	Pristupna točka za uređaje, Pristupna točka za korisničko sučelje	socket	Događaj generiran pri povezivanju uređaja ili korisničke aplikacije
disconnect	Uređaji (automatski)	Pristupna točka za uređaje	-	Događaj generiran pri odspajanju uređaja
request-data	Korisničko sučelje	Pristupna točka za korisničko sučelje	String(id)	Korisnički zahtjev za trenutnim slanjem podataka
devices-update	Pristupna točka za korisničko sučelje	Korisničko sučelje	uredaji	Promjena u popisu povezanih uređaja
new-data	Pristupna točka za korisničko sučelje	Korisničko sučelje	podaci	Pristizanje novih podataka direktno, bez prethodne pohrane
database_data	Pristupna točka za korisničko sučelje	Korisničko sučelje	podaci	Pristizanje novih podataka koji su

				prethodno pohranjeni u bazu podataka
--	--	--	--	---

5.5.3. Pregled krajnjih točaka za http zahtjeve

U Tablici 5.5. prikazane su HTTP krajnje točke korištene za komunikaciju u sustavu. Dan je pregled adresa krajnjih točaka, pošiljatelja i primatelja zahtjeva, korištene HTTP metode te kratak opis uloge svake pojedine krajnje točke.

Tablica 5.5. Pregled HTTP krajnjih točaka zajedno s kontekstom unutar sustava

HTTP krajnja točka	HTTP metoda	Pošiljatelj zahtjeva	Primatelj zahtjeva	Opis
http://localhost:3000/podaci	GET	Korisničko sučelje	Pristupna točka za korisničko sučelje	Dohvat svih podataka iz baze, u ovom slučaju povijesnih podataka
http://localhost:3000/podaci	DELETE	Korisničko sučelje	Pristupna točka za korisničko sučelje	Brisanje svih podataka iz baze

5.6. Implementacijska ograničenja

Prototip izrađen u sklopu ovog rada simulira osnovne mehanizme prijenosa poruka i upravljanja uređajima, no sadrži i određena ograničenja. Spomenuta ograničenja variraju po prirodi i važnosti. Jedno od očitih ograničenja je nedostatak trajne pohrane podataka na razini posrednika poruka. Naime, u implementaciji je korišten NATS posrednik, bez JetStreama, koji bi omogućio trajnu pohranu poruka. Trajnost se postiže isključivo bazom podataka, što je

dovoljno kako bi simulator ispunjavao svrhu i pokazao željeno ponašanje, no ne osigurava u potpunosti od gubitka poruka u sustavu. Ako bi iz bilo kojeg razloga došlo do gubitka poruke nakon njezine objave na NATS predmet, moglo bi se dogoditi da željena komponenta nikad ne primi poruku te bi došlo do njezinog gubitka.

Još jedno ograničenje ove implementacije je nedostatak sigurnosnih mehanizama autentifikacije i autorizacije. Prilikom slanja bilo NATS poruka, bilo Socket.io događaja, ne postoji mehanizam koji bi utvrdio šalje li poruku komponenta predviđena implementacijom, a isto vrijedi i za primanje poruke. Treća strana bi se, npr., mogla povezati u sustav, predstavljajući se kao uređaj te slati vlastite podatke ili objavlјivati/pretplatiti se na NATS predmet i tako ometati rad sustava. S ovim je povezano i ograničenje glede nedostatka sanitizacije ulaznih podataka.

U sustavu postoji i nedostatak glede jedinstvenosti ID vrijednosti uređaja. Naime, implementacija trenutno nalaže da korisnik sam određuje i unosi ID vrijednosti uređaja kojima se povezuje u sustav. To znači da je osiguravanje jedinstvenosti tih ID vrijednosti zadaća korisnika ili skripte koja pokreće uređaje. U slučaju pokretanja više uređaja iste ID vrijednosti, može doći do nepredviđenog i neispravnog ponašanja sustava.

Trenutna arhitektura implementiranog rješenja ograničena i u vidu horizontalnog skaliranja. Trenutno se pokreće samo jedna instanca svake poslužiteljske komponente, što omogućuje nesmetano korištenje sustava u uvjetima testiranja, no u slučaju velikog broja uređaja i korisnika, vjerojatno ne bi zadovolјilo propisane nefunkcionalne zahtjeve te performanse ne bi bile u skladu s industrijskim standardima. Kako bi se ovaj problem riješio, bilo bi potrebno pokrenuti veći broj komponenti poslužitelja te implementirati komponentu za ujednačavanje opterećenja (engl. *load balancing*), koja bi pristigle podatke te korisničke zahtjeve raspoređivala po različitim instancama navedenih komponenti. To bi efektivno podrazumijevalo prelazak na istinski distribuiranu arhitekturu. Trenutni koncept podržava ovakvu nadogradnju pa je to jedan od sljedećih logičnih koraka poboljšanja razvijenog sustava.

Svakako valja spomenuti i ograničenje nastalo korištenjem besplatne opcije na Renderu u svrhu puštanja baze podataka u pogon. Korištenje te opcije ograničava kapacitet, dostupnost te performanse baze podataka. Ovo ograničenje, kao i prethodno, ne dolazi do izražaja u

okruženju ispitivanja, no u produkcijskom okruženju, trenutne postavke baze podataka vjerojatno ne bi bile dovoljne za osiguravanje potrebnih performansi.

U ovom poglavlju prikazana je implementacija prototipa kroz simulator izvora događaja, poslužiteljske komponente, korisničko sučelje, podatkovni model i komunikacijske ugovore. Posebno je istaknuto razdvajanje brzog toka podataka prema korisničkom sučelju od toka kojim se potvrđuje trajna pohrana u bazu. Takav pristup omogućuje nisku latenciju prikaza, ali istodobno pruža korisniku informaciju o tome je li događaj potvrđeno spremljen.

Opisani komunikacijski ugovori i vremenske oznake u bazi stvaraju osnovu za evaluaciju prototipa. Time se poglavlje 5 povezuje s poglavljem koje slijedi, u kojem se ponašanje sustava može provjeriti kroz mjerila latencije, cjelovitosti obrade, pojave duplikata i ponašanja pri različitim opterećenjima.

6. Evaluacija prototipa

7. Zaključak

U ovom radu oblikovan je i implementiran modularni prototip za obradu događaja u blizu stvarnom vremenu. Dan je pregled osnovnih pojmova i koncepata iz područja distribuiranih sustava. Opisani su i uspoređeni različiti posrednici poruka te je posebno detaljno opisan NATS, kao tehnologija korištena u ovom radu. Opisano je ostvarivanje dvosmjerne komunikacije putem WebSocketeta i Socket.io protokola. Razvijen je prototip koji demonstrira funkcionalnosti generiranja, primanja i obrade događaja korištenjem navedenih tehnologija. Prikazani su dijelovi koda važni za razumijevanje rada razvijenog programa te su povezani s prethodno definiranim zahtjevima implementacije. Na kraju su prikazana i ograničenja samog sustava s objašnjenjem kako ona negativno utječu na korištenje.

Izrađeni prototip obuhvaća simulatore izvora događaja, pristupnu točku za uređaje, sloj obrade i pohrane podataka, pristupnu točku za korisničko sučelje, web sučelje za nadzor događaja i PostgreSQL bazu podataka. Simulatori uređaja implementirani su u programskom jeziku Python i omogućuju pokretanje većeg broja izvora događaja u ispitnom okruženju. Poslužiteljske komponente implementirane su u okruženju Node.js, dok se njihova međusobna komunikacija ostvaruje preko NATS predmeta. Rezultat rada jest razdvajanje brzog toka prikaza od toka potvrđene pohrane. Neposredno po primitku događaja, podatak se može proslijediti korisničkom sučelju radi smanjenja latencije prikaza. Nakon toga se isti događaj obrađuje u sloju pohrane i zapisuje u bazu podataka, a korisničkom sučelju se šalje potvrđena kopija zapisa. Time se korisniku može prikazati razlika između događaja koji je samo zaprimljen i događaja koji je trajno pohranjen. Takav pristup izravno odgovara zahtjevu za brzim odzivom, ali i potrebi za naknadnom provjerom cjelovitosti obrade.

Nedostaci implementacije opisani u radu predstavljaju temelj za daljnje poboljšanje prototipa te unaprjeđenje njegove funkcionalnosti. Posebno važan status ima horizontalno skaliranje, kojim bi se mogućnosti i kapacitet sustava podigli na mnogo višu razinu te bi dobiveno rješenje uvelike simuliralo rad stvarnog produkcijskog sustava za obradu događaja. Prije korištenja sustava u stvarnom okruženju, nužno bi, dakako, bilo i riješiti nedostatke vezane za sigurnost sustava, posebno za autentifikaciju i autorizaciju, kako bi se spriječio neovlašteni pristup te manipulacija sustavom i događajima.

LITERATURA

- [1] „Core concepts. CAP Theorem. Master the fundamental tradeoffs between consistency and availability in distributed systems.“, Hello Interview. [Online]. Dostupno: <https://www.hellointerview.com/learn/system-design/core-concepts/cap-theorem>
[Pristupljeno: 19.6.2026.]

[2] „What is an Event-Driven Architecture? Decoupled systems that run in response to events“, AWS Amazon. [Online]. Dostupno: <https://aws.amazon.com/event-driven-architecture/> [Pristupljeno: 25.6.2026.]

[3] „Event-Driven Architecture“, GeeksForGeeks. [Online]. Dostupno: <https://www.geeksforgeeks.org/system-design/event-driven-architecture-system-design/> [Pristupljeno: 25.6.2026.]

[4] „Event Time vs. Processing Time in Data Engineering“, Medium. [Online]. Dostupno: <https://medium.com/@senthilcegpotech/event-time-vs-processing-time-in-data-engineering-f599d0373fdd> [Pristupljeno: 25.6.2026.]

[5] Leslie Lamport, *Time, Clocks and the Ordering of Events in a Distributed System*. [Online]. Dostupno: <https://lamport.azurewebsites.net/pubs/time-clocks.pdf> [Pristupljeno 26.5.2026.]

[6]

[7]

[8]

[9]

[10]

[11]

[12]

[13]

[14]

[15]

[16]

[17]

[18]

[19]

[20]

[21]

[22]

[23]

[24]

[25]

[26]

[27]

[28]

[29]

[30]

[31]

[32]

Sažetak

Naslov: Modularni sustav za obradu događaja u stvarnom vremenu s gamifikacijom

U ovom radu obrađen je problem obrade događaja koji pristižu u sustav iz više izvora u blizu stvarnom vremenu. Cilj rada bio je razviti modularni sustav koji omogućava prihvatanje, obradu, trajno pohranjivanje te prikazuje korisniku podatke generirane na skupu uređaja. Predloženo rješenje zasniva se na događajno vođenoj arhitekturi u kojoj se komunikacija između komponenti ostvaruje putem NATS posrednika poruka, a dvosmjerna komunikacija s uređajima i korisničkim sučeljem ostvaruje pomoću Socket.io protokola. Funkcionalnost sustava je provjerena ispitivanjem latencije i rada sustava u nizu različitih ispitnih slučajeva, koji obuhvaćaju uvjete od najjednostavnijih do zahtjevnih, s visokim opterećenjem, brzim pristizanjem podataka i velikim brojem uređaja. Ovime se stječe uvid u skalabilnost i propusnost sustava, što je nužno za određivanje uspješnosti i kvalitete implementacije samog rješenja.

Ključne riječi: događajno vođena arhitektura; latencija; NATS; obrada događaja

Summary

Title: Modular real-time event processing system with gamification

This thesis addresses the problem of processing events that arrive into the system from multiple sources in near real-time. The goal of the thesis was to develop a modular system which enables acquisition, processing, persistent storage and presentation of the data generated by the device set to the user. Proposed solution is based on event-driven architecture, in which communication between components is achieved using NATS message broker and two-way communication with devices and the user interface is achieved using Socket.io protocol. System functionality was evaluated by testing the latency and the system operation in the array of different test cases, from the simplest, to demanding ones, with high load, rapid data arrival and large number of devices. This way, an insight into the scalability and throughput of the system is gained, which is necessary for determining success and quality of the implementation of the solution itself.

Keywords: event-driven architecture; latency; NATS, event processing

Dodatak A: Elektronički dodaci i izvorni kod

Cjelovito programsko rješenje opisano u ovom radu dostupno je putem GitHub repozitorija na poveznici: [URL](#) . Repozitorij sadrži cjelovit programski kod, uključujući kod poslužiteljskih komponenti, kod korisničkog sučelja te kod simulatora uređaja, zajedno s uputama za pokretanje sustava. Prije početka rada, potrebno je napisati vlastitu konfiguracijsku datoteku okruženja, kako bi se omogućilo povezivanje na proizvoljnu bazu podataka. Prethodno je potrebno i kreirati tablicu podaci, pokretanjem SQL naredbe prikazane Isječkom 5.16.

Dodatak B: NATS primjeri koda i način korištenja

B.1. Korištenje NATS-a u Node.js okruženju

Brojni programski jezici i alati podržavaju rad s NATS poslužiteljem [29]. Jedan od programskih jezika u kojima se NATS često koristi je JavaScript, a posebno u kontekstu poslužiteljske strane aplikacija (engl. *backend*). Node.JS je uobičajeno korišten alat za ostvarivanje poslužiteljske strane aplikacija u modernom programskom inženjerstvu. Kako bi se omogućilo korištenje NATS funkcionalnosti u sklopu Node.JS-a, najprije je potrebno preuzeti NATS paket korištenjem naredbe `npm install nats`. Rad s NATS-om započinje pozivom funkcije `connect`, čime se uspostavlja veza s poslužiteljem. Navedena funkcija vraća objekt razreda `NatsConnection`, koji će, zatim, služiti za izvođenje operacija s porukama korištenjem NATS poslužitelja. Nakon završetka rada s poslužiteljem, konekcija se prekida pozivom funkcije `close()`. Ovo je osnovni način prekida konekcije koji konekciju odmah prekida. Međutim, postoji i funkcija `drain()`, koja će najprije obraditi sve poruke koje se još nalaze u sustavu, a tek zatim zatvoriti konekciju. Korištenjem ove funkcije se smanjuje mogućnost gubitka poruke.

```
import { connect } from "nats";  
const konekcija = await connect({ servers: "nats://localhost:4222" });  
// konekcija.close(); nesigurno zatvaranje  
// konekcija.drain(); sigurno zatvaranje
```

Osnovna NATS funkcionalnost objave i pretplate ostvarena je funkcijama `publish` i `subscribe`. Funkcija `publish` kao prvi argument prima naziv predmeta na koji se poruka šalje, a drugi argument predstavljaju sami podaci poruke. S obzirom da se podaci u NATS protokolu šalju kao niz bajtova, tj. `Uint8Array`, potrebno ih je najprije kodirati prije slanja. Također, nakon primanja poruke, primatelj dobivene bajtove dekodira, kako bi bio u stanju čitati i

koristiti podatke. Kodiranje i dekodiranje ostvaruje se funkcijama `encode()` i `decode()`. Ovisno o tipu podataka koji se šalju, za kodiranje i dekodiranje koristi se određeni kodek (engl. *codec*). Za JSON objekte se, tako, koristi `JSONCodec`, dok se za nizove znakova (engl. *string*) koristi `StringCodec`. Najprije je potrebno istoimenim funkcijama konstruktorima kreirati objekt željenog kodeka, a zatim, korištenjem istog, kodirati željeni objekt prije slanja poruke.

```
const jc = JSONCodec();
const sc = StringCodec();
konekcija.publish("predmet.potpredmet", jc.encode({id: 1, ime: "Ante",
  prezime: "Kolak"}));
konekcija.publish("predmet.potpredmet2", sc.encode("Poruka u obliku
  stringa"));
```

Komponenta sustava koja treba primiti poslane poruke mora se pretplatiti na odgovarajući predmet ili više njih, što se postiže funkcijom `subscribe`. Ta funkcija vraća objekt po kojem se može iterirati i koji sadrži poruke poslane na navedeni predmet. Asinkronim iteriranjem po tom objektu, dobivaju se poruke iz njega te se ujedno i čeka na svaku buduću poruku, koja se, po pristizanju, automatski i prima te može dalje koristiti. Primjer navedene funkcionalnosti prikazan je u sljedećem kodu:

```
const pretplata = konekcija.subscribe("predmet.potpredmet");
for await (const message of pretplata) {
  console.log(jc.decode(message.data));
}
```

Predmeti na koje se moguće pretplatiti obično se definiraju hijerarhijski, po razinama. Prilikom definiranja predmeta, razine se odvajaju znakom točke. Funkcija `subscribe` omogućuje istovremenu pretplatu na više povezanih predmeta uporabom posebnih simbola.

Tako npr. simbol „>“ označava da se pretplata odnosi na sve potpredmete do tada definiranog predmeta. Predmeti obuhvaćeni uporabom ovog simbola uključuje sve podrazine, bez obzira na dubinu. Ako su predmeti u nekom sustavu definirani prema strukturi:

`zgrada.kat.hodnik.prostorija.uredjaj`, pretplata na temu `zgrada1.kat2.>` podrazumijevala bi pretplatu na teme vezane za sve hodnike, sve prostorije i sve uređaje na drugom katu zgrade 1. Simbol „*“, za razliku od prethodnog simbola, zamjenjuje samo jedan sloj hijerarhije predmeta. Tako bi pretplata na temu `zgrada1.*.hodnik5.>` podrazumijevala pretplatu na predmete vezane za sve prostorije i sve uređaje hodnika 5 na svakom katu zgrade 1. Ovakav identifikator predaje se u obliku stringa, kao argument funkciji `subscribe`.

```
const sub = konekcija.subscribe("zgrada1.*.hodnik5.>");
```

Funkcionalnost zahtjeva i odgovora ostvarena je funkcijama `request` i `respond`. Funkcija `request` slična je funkciji `publish`. Poziva se s ključnom riječju `await` te se, prilikom njezinog poziva, može postaviti vrijednost `timeout`, koja određuje koliko će se dugo čekati na odgovor u milisekundama. Povratna vrijednost ove funkcije je obećanje (engl. *promise*), a po primitku odgovora, dobiva se objekt koji predstavlja dobiveni odgovor i koji se, zatim, dekodira i iz kojeg se, nakon toga, dobivaju potrebni podaci. Komponenta kojoj je zahtjev upućen, prima ga kao i svaku drugu poruku, tj. pretplatom na odgovarajući predmet. Nakon primanja poruke, komponenta poziva funkciju `respond` nad objektom poruke. Kao argument, ta funkcija prima željeni sadržaj, kodiran odgovarajućim kodekom. Kodirani sadržaj automatski se šalje komponenti koja je poslala zahtjev, kao što je već opisano. U sljedećem kodu prikazan je primjer ove funkcionalnosti u kojem pošiljalac u zahtjevu šalje djeljenik i djelitelj, primatelj prima zahtjev i odgovara rezultatom dijeljenja:

```
// Kod pošiljatelja
const odgovor = await konekcija.request("predmet", jc.encode({a: 100, b: 2})),
    { timeout: 5000 });
```

```
// Kod primatelja
const pretplata = konekcija.subscribe("predmet");
for await(const poruka of pretplata) {
    const {a, b} = jc.decode(poruka.data);
    poruka.respond(jc.encode({rezultat: a / b})) ;
}
```

Funkcionalnost grupa reda postiže se navođenjem vrijednosti parametra `queue` prilikom pretplaćivanja na predmet. Sve komponente koje se pretplate koristeći istu vrijednost tog parametra postaju dio iste grupe reda te će samo jedna od tih komponenti, ona nasumično odabrana, primiti poruku. Može postojati proizvoljan broj grupa reda na svakom predmetu.

```
konekcija.subscribe("predmet.potpredmet", {queue: "grupa1"});
```

Korištenje JetStream funkcionalnosti započinje kreiranjem objekta JetStream managera. Nakon toga je moguće kreirati tok dodavanjem novog toka atributu `.streams` tog objekta. Dodavanje toka postiže se metodom `.add()`, kojoj se, kao argument, predaje konfiguracijski objekt koji sadrži željene parametre toka. Ti parametri određuju glavne karakteristike toka, popu vremena trajanja, kapaciteta i sl. Jedan primjer kreiranja JetStream toka prikazan je u sljedećem isječku koda:

```
const manager = await konekcija.jetstreamManager();
const konfiguracija = {
    name: "UPLATE",
    subjects: ['uplate.>'],
    retention: 'limits',
    storage: 'file',
    max_msgs: 500000,
```



```

    max_bytes: 1024 * 1024 * 1024,
    num_replicas: 1,
    duplicate_window: 180000000000 //3 minute u nanosekundama
};

```

```

await manager.streams.add(konfiguracija);

```

Atribut `name` konfiguracijskog objekta predstavlja ime toka i često se piše velikim tiskanim slovima. Atribut `subjects` predstavlja sve predmete čije poruke će biti dostupne u toku. I ovdje je moguće korištenje posebnih simbola za označavanje više predmeta odjednom. Atribut `retention` određuje do kad se poruke zadržavaju u toku. Ako se vrijednost tog atributa postavi na `"limits"`, kao što je u prethodnom primjeru, tada se podaci u toku zadržavaju dok god granice kapaciteta ili trajanja toka nisu prijeđene. Ako se koristi vrijednost `"interest"`, poruke se čuvaju dok god postoje konzumeri koji ih čitaju. Drugim riječima, poruke se brišu kad više nisu predmet interesa ni jednog konzumera. Da bi se to dogodilo, svi konzumeri koji trenutno čitaju s predmeta na koji je poslana ta poruka moraju potvrditi primitak. Ako neka poruka pristigne u trenutku kad nema niti jednog konzumera, ona se odmah briše. Ako se koristi vrijednost `"workqueue"`, tada se poruka briše nakon potvrđenog primanja. Ovime se znatno smanjuje broj poruka u toku jer se ne čuvaju stare poruke, a istovremeno se osigurava primanje poruke. Zato je ovaj pristup memorijski manje zahtjevan od ostalih. Međutim, robusnost sustava ipak je niža jer, ako nekoj drugoj komponenti kasnije bude potreban isti podatak, on će morati biti poslan ponovno. Po tome je ovakav pristup sličan redu poruka, baš kao što sam naziv govori. Činjenica da se poruka briše nakon čitanja, povlači za sobom posljedicu da samo jedan potrošač može pročitati svaku poruku. Kako bi se osigurao ispravan rad sustava, potrošači moraju imati vrijednosti parametra `filter_subject` koje se ne preklapaju, tj. disjunktne. Značenje ovog parametra opisano je kasnije u tekstu. Atribut `storage` određuje vrstu pohrane podataka u toku. Taj atribut može poprimiti vrijednosti `"file"` i `"memory"`. Ako se odabere *file* način pohrane, podaci se spremaju trajno, kao na disk. Ako se odabere *memory* način pohrane, podaci se brišu iz reda prilikom ponovnog pokretanja, kao u radnoj memoriji. Atribut `max_msgs` označava maksimalan kapacitet toka u pogledu poruka, odnosno maksimalan broj poruka koje se mogu istovremeno

nalaziti u toku. Atributom `max_bytes` zadaje se ograničenje veličine u pogledu bajtova. Kad se bilo koje od ova dva ograničenja veličine dostigne, tok se smatra popunjenim, i u slučaju potrebe za dodavanjem novih poruka u tok, JetStream oslobađa prostor uklanjanjem starijih poruka prema definiranim pravilima. `max_age` parametar uvjetuje maksimalni rok trajanja poruke, nakon čijeg isteka se poruka automatski briše. Zadaje se u nanosekundama. Parametar `num_replicas` određuje koliko replika samog toka će postojati. Ova funkcionalnost posebno je korisna u slučajevima potrebe za visokom dostupnosti, kad nije prihvatljivo da poruku u nekom trenutku nije moguće poslati u tok. Naime, u slučaju postojanja nekoliko replika istog toka, nedostupnost jedne replike nadomještena je postojanjem ostalih ekvivalentnih replika. Nakon eliminacije problema i ponovnog uspostavljanja veze s replikom toka s kojom je komunikacija bila prekinuta, događa se sinkronizacija, kojom se osigurava da su sve replike toka ekvivalentne po sadržaju. Ipak, takav pristup memorijski je zahtjevniji, a u slučaju velikih tokova s velikom količinom poruka koje moraju dugo trajati te u slučaju postojanja mnogo tokova u sustavu od kojih svi moraju imati svoje replike, ovaj problem može postati značajan. Parametar `duplicate_window` navodi se u nanosekundama i označava vremenski period unutar kojeg se provjerava je li došlo do ponovnog slanja iste poruke. Ako se ovaj parametar postavi na nulu, provjera duplikata se ne provodi.

Objavljivanje poruke prilikom korištenja JetStreama, razlikuje se od uobičajenog po tome što se poruke pohranjuju u tok. Zato `publish()` metoda u sklopu JetStreama vraća objekt koji služi kao potvrda uspješnog dodavanja. Povratna vrijednost se dobiva tek nakon uspješne pohrane poruke u tok, a dobiveni objekt sadrži atribut koji pružaju dodatne informacije o pohranjenoj poruci, između ostaloga naziv toka (atribut `.stream`) i redni broj poruke u tom toku (atribut `.seq`).

Potrošač (engl. *consumer*), kreira se uporabom metode `add`, koja se poziva nad atributom `consumers` objekta JetStream menadžera kreiranog na početku. Prilikom poziva metode `add`, kao prvi argument navodi se naziv toka na koji želimo prikvačiti potrošača, a kao drugi argument postavlja se konfiguracijski objekt koji definira svojstva i karakteristike tog potrošača. To se, primjerice, može učiniti na sljedeći način:

```
await manager.consumers.add("UPLATE", {
    durable_name: "konzumer1",
    ack_policy: AckPolicy.explicit, //ili AckPolicy.none ili AckPolicy.all
    deliver_policy: DeliverPolicy.All,
    max_deliver: 4,
    max_ack_pending: 250,
    ack_wait: 60000000000,
    filter_subject: "predmet.potpredmet1"
});
```

U konfiguracijskom objektu u prethodnom programskom odsječku, parametar `durable_name` definira ime potrošača. Potrošač koji ima definirano ime je trajni potrošač (engl. *durable consumer*) te on pamti svoju poziciju i nastavlja nakon ponovnog spajanja. Ako se ovaj parametar izostavi, nastaje privremeni potrošač (engl. *ephemeral consumer*), koji se, nakon odspajanja, briše i nestaje s poslužitelja. Ako se potrošač ne kreira eksplicitno, ispod haube se zapravo stvara upravo privremeni potrošač. Parametar `ack_policy` određuje treba li primatelj poruke potvrditi njezino primanje. Ako se vrijednost ovog parametra postavi na `"explicit"`, potrebno je posebno potvrditi svaku poruku. Ako se vrijednost postavi na `"all"`, potrebno je potvrditi samo posljednju poruku u nizu, a time se ostale poruke automatski smatraju potvrđenima. U slučaju postavljanja vrijednosti `"none"`, nema potrebe za potvrđivanjem primitka poruke. Parametar `deliver_policy` određuje odakle potrošač počinje čitati tok. Najčešće se koristi opcija `"all"`, koja podrazumijeva čitanje od prve poruke toka. Međutim, moguće je odabrati i opciju `"new"`, kojom se čitaju isključivo nove poruke, tj. one pristigle nakon kreiranja potrošača, te opciju `"last"`, koja čita isključivo zadnju poruku u toku. Ta se opcija razlikuje od opcije `"new"` utoliko što čita i zadnju poruku koja je već bila prisutna u toku u trenutku kreiranja potrošača, a zatim čeka na nove poruke, kao i s opcijom `"new"`. Postoje i opcije `"by_start_time"` i `"by_start_sequence"`, koje omogućuju točno određivanje početne pozicije s koje potrošač počinje čitati. Naime, `"by_start_time"` govori da čitanje počinje od točno određenog vremena. Korištenje ove opcije zahtijeva dodatni parametar `opt_start_time`, koji kao vrijednost ima zapis datuma i vremena u obliku stringa. Opcija

“by_start_sequence” definira početak čitanja od točno određenog rednog broja poruke u toku. Korištenje ove opcije zahtijeva navođenje dodatnog parametra `opt_start_seq`, čija je vrijednost prirodni broj, koji označava redni broj početne poruke. Parametar `max_deliver` označava koliko će se puta pokušati poslati poruku, u slučaju da se ne primi potvrda primanja. Za to je vezan i parametar `ack_wait`, koji određuje koliko dugo se čeka na potvrdu primanja u nanosekundama. Nakon isteka zadanog vremena, poruka se ponovo šalje. Ciklus se ponavlja najviše onoliko puta koliko je određeno parametrom `max_deliver`. `max_ack_pending` je parametar koji predstavlja najveći dozvoljeni broj nepotvrđenih poruka. Valja naglasiti da ovo ne znači da, ako se taj maksimalni broj dostigne, neće biti moguće slati nove poruke u red. Ovo jednostavno znači da potrošač neće uzimati dodatne poruke u obzir, dok će neki drugi potrošač s višim ograničenjem moći pročitati tu poruku. Parametar `filter_subject` omogućuje potrošaču da uzima u obzir samo poruke koje su poslane na željeni predmet/predmete. Ovo je iznimno korisna funkcionalnost, s obzirom da se u tokovima obično nalaze poruke poslane na više različitih predmeta.

Primanje podataka putem potrošača može se ostvariti dvama načinima : guranjem (engl. *push*) te vučenjem (engl. *pull*). Guranje podrazumijeva kontinuirano i automatsko primanje poruka kroz uspostavljeni kanal, pri čemu sam poslužitelj obavlja radnju slanja, dok je klijent pasivan te samo prima poruke. Vučenje je pristup u kojem klijent ima aktivnu ulogu i traži podatke eksplicitno, slanjem zahtjeva poslužitelju. Primanje poruka temeljeno na guranju ostvaruje se korištenjem metode `consume()`, a ono temeljeno na vučenju koristi metodu `fetch()`. Kao argument metode `fetch` može se navesti broj poruka koje je potrebno primiti odjednom. Primanje poruka temeljeno na guranju prikladnije je za situacija u kojima se velika količina podataka kontinuirano šalje potrošaču, no uz rizik preopterećenja potrošača, ovisno o količini i volumenu poruka. Primanje poruka temeljeno na vučenju prikladnije je u slučaju potrošača ograničenog kapaciteta i ograničene brzine obrade poruka, kad poruke pristižu neravnomjerno i kad se potrošaču treba dopustiti kontrola i odabir trenutka primanja poruke. Primjer ostvarivanja navedenih funkcionalnost prikazan je u sljedećem odsječku koda :

```
const js = konekcija.jetstream() ;  
const consumer = await js.consumers.get("UPLATE", "konzumer1");
```

```
//Push način primanja poruka (consume):

const poruke = await consumer.consume();
for await (const poruka of poruke) {
    podaci_dekodirani = jc.decode(poruka.data);
    // Daljnje operacije
    poruka.ack() ;
}

// Pull način primanja poruka (fetch)

const poruke = await consumer.fetch({max_messages: 5});
for await (const poruka of poruke) {
    podaci_dekodirani = jc.decode(poruka.data);
    // Daljnje operacije
    poruka.ack();
}
```

U prethodnim primjerima, metoda `.ack()` predstavlja potvrdu primitka poruke. To je upravo potvrda na koju se odnosi `ack_wait` atribut potrošača. Ako u navedenom intervalu ne stigne potvrda, poruka se iznova šalje. Osim potvrde primanja putem `poruka.ack()`, moguće je izvršiti i metodu `poruka.nak()`, što znači da obrada nije uspjela te se zahtijeva ponovno slanje poruke. Ako se navedenoj metodi preda argument koji predstavlja broj milisekundi, to označava da obrada poruke nije uspjela, no da se ponovno slanje izvrši s odgodom koja odgovara predanoj vrijednosti. Ako se, pak, izvrši `poruka.term()`, to je znak da se odustane od daljnjeg slanja poruke. U tom slučaju, ne pokušava se ponovo poslati poruku, čak i ako `max_deliver` vrijednost još nije dostignuta. Ako je izgledno da će obrada poruke biti uspješna te novo slanje neće biti potrebno, no obrada traje dulje od `ack_wait` intervala (pa bi poruka inače bila ponovo poslana unatoč izostanku stvarne potrebe za tim), može se koristiti naredba `poruka.working()` kako bi se navedeno signaliziralo te interval čekanja produljio.

Funkcionalnost pohrane ključeva i vrijednosti također je implementirana nad tokovima podataka koje omogućuje Jetstream. Navedena funkcionalnost slična je rječnicima, a sučelje za rad s njom također je ekvivalentno rječničkim sučeljima. Moguće je dodavanje, uređivanje i brisanje parova ključ-vrijednost te dohvat vrijednosti pridružene zadanom ključu. Prilikom dodavanja para ključ-vrijednost, vrijednost se kodira potrebnim kodekom, baš kao i kod slanja uobičajenih NATS poruka. Dodavanje se vrši uporabom metode `put(ključ, vrijednost)`, a ista se metoda koristi i za promjenu vrijednosti pridruženu već postojećem ključu. Vrijednost pridružena željenim ključu dobiva se metodom `get(ključ)`. Zapis se briše uporabom `delete(ključ)`. Primjer navedenoga prikazan je u sljedećem programskom odsječku:

```
const js = nc.jetstream();
const kv = await js.views.kv("konfiguracija");

await kv.put("kljuc1", sc.encode("vrijednost1"));
const vrijednost = await kv.get("kljuc1");
vrijednost_dekodirana = sc.decode(vrijednost.value);
await kv.delete("kljuc1");
```

U sklopu ključ-vrijednost funkcionalnosti, dostupna je i funkcija `watch()`. Ona omogućuje reagiranje na promjene vrijednosti nekog ključa (ili više njih) u realnom vremenu. Osim definiranja jednog konkretnog promatranog ključa, moguće je i korištenje specijalnih simbola za definiranje skupine ključeva. Ovo je posebno korisno ako se ključevi imenuju hijerarhijskim razinama odvojenim točkama ili nekom drugom sličnom konvencijom. Poziv te funkcije daje novu vrijednost svaki put kad dođe do promjene vrijednosti promatranog ključa. Primjer jednostavnog korištenja funkcije `watch()` prikazan je u nastavku:

```
const gledatelj = await kv.watch({key: "kljuc1"});
for await (const zapis of gledatelj) {
  console.log(zapis.key);
}
```

```
    console.log(sc.decode(zapis.value));  
  }  
}
```

Ono po čemu se sustav ključeva i vrijedosti Jetstreama ipak razlikuje od uobičajenog je činjenica da se svaka promjena rječnika zapisuje u tok kao zasebna poruka. To omogućava pregled povijesti svih promjena rječnika, tj. barem onoliko koliko ograničenja samog toka dopuštaju.

Korištenje funkcionalnosti *Object Store* po sučelju je vrlo slično sustavu ključeva i vrijednosti. Naime, nazivi korištenih metoda također su `put`, `get` i `delete`. Najprije se kreira *Object Store* objekt na sljedeći način:

```
const os = await js.views.os("datoteke");
```

Pohrana objekata/datoteka u spremnik izvodi se naredbom `put`. Ta naredba, kao prvi argument, prima ime pod kojim spremamo određeni objekt (pandan ključu u spremniku ključeva i vrijednosti), dok je drugi argument binarni zapis podataka u datoteci koju spremamo, tj. njezin sadržaj:

```
await os.put({name: "katastarski_plan.pdf"}, buff);
```

Dohvat podataka iz spremnika postiže se metodom `get`:

```
const datoteka = await os.get("katastarski_plan.pdf");
```

Brisanje objekata/datoteka moguće je postići korištenjem metode `delete()`. Time se referencirani objekt trajno uklanja iz spremnika:

```
await os.delete("katastarski_plan.pdf");
```

Sam *Object Store* objekt `os` omogućava pristup popisu svih pohranjenih objekata. On se može dohvatiti pozivom `os.list()`, pri čemu je povratna vrijednost te metode obećanje (engl. *Promise*). Razrješavanjem tog promisa dobiva se *Async Iterable* objekt, tj. objekt po kojem se može iterirati, no čiji elementi nisu odmah eksplicitno dostupni, već se po potrebi razrješavaju uz korištenje ključne riječi `await`. Elementi dobivenog iterabilnog objekta ne sadrže binarni zapis datoteke, već njezine metapodatke, kao što su ime ili veličina.

```
const iterable = await os.list() ;    //iterabilni objekt
for await (const element of iterable) {    //jedan element za svaki objekt
  console.log(element.size);
    console.log(element.name);
}
```


Dodatak C: WebSocket i Socket.io primjeri koda i način korištenja

C.1. Websocket u programskom jeziku Javascript

Za rad s osnovnim WebSocket aplikacijskim programskim sučeljem (engl. *Application programming interface, API*), postoje naredbe u mnogim programskim jezicima [30]. U nastavku opisane su osnovne naredbe u programskom jeziku JavaScript. Najprije se kreira WebSocket objekt korištenjem konstruktora, pri čemu se ujedno odvija i već opisano rukovanje. Kreiranjem navedenog objekta, započinje proces rukovanja, nakon kojeg je veza uspostavljena i spremna za korištenje.

```
const ws = new WebSocket("ws://localhost:8080");
```

Poruke se šalju izvršavanjem metode `send()`, kojoj se, kao argument, predaje sadržaj poruke. Sadržaj se najčešće šalje u obliku stringa, a u slučaju slanja JSON objekta, on se može pretvoriti u string metodom `JSON.stringify()`.

```
ws.send("Poruka 1 za poslužitelj");  
const objekt_json_formata = {atr1: 1, atr2: 2};  
ws.send(JSON.stringify(objekt_json_formata));
```

Osim pukog slanja poruka, moguće je definirati i slušatelje događaja (engl. *event listener*), koji se aktiviraju automatski u slučaju određenog događaja te izvršavaju zadani kod. Događaji za koje je moguće definirati slušatelje su: `open`, `close`, `message` i `error`. Događaj „`open`“ izvršava se odmah po uspostavljanju WebSocket komunikacije. Događaj „`close`“ nastupa po

zatvaranju veze. Događaj „message“ nastupa pri primanju poruke, dok „error“ podrazumijeva pogrešku u vezi. Primjer definiranja slušatelja događaja za događaj „open“ prikazan je u sljedećem kodu:

```
ws.onopen = () => {  
  console.log("Veza uspostavljena!");  
  ws.send("Hello world!");  
};
```

Ovaj pristup je elegantan, no omogućava dodavanje samo jednog slušatelja događaja za pojedini događaj. Ako je potrebno imati više različitih slušatelja istog događaja, moguće ih je dodati proizvoljan broj koristeći metodu `addEventListener`:

```
ws.addEventListener("open", () => console.log("Drugi listener"));  
ws.addEventListener("open", () => console.log("Treći listener"));
```

Iako je WebSocket već sam po sebi moćan protokol, on pruža samo osnovne komunikacijske funkcionalnosti, dok mogućnosti poput dodavanja klijenata u „sobe“, kreiranja proizvoljnih događaja proizvoljnog imena i sl. nisu izvorno implementirane. Te i mnoge druge funkcionalnosti dostupne su u sklopu Socket.io biblioteke opisane u sljedećem poglavlju.

C.2. Socket.io u Node.js okruženju

Okruženje Node.js i programski jezik JavaScript uobičajeni su alati koji nude mogućnost rada sa Socket.io bibliotekom [31]. Ovisno o potrebama, u sklopu Node.js-a moguće je ostvariti Socket.io poslužitelj, Socket.io klijent ili oboje. Navedeni dijelovi omogućuju se preuzimanjem odgovarajućih *npm* paketa. Poslužiteljski dio instalira se

naredbom: `npm install socket.io`, a klijentski dio naredbom `npm install socket.io-client`.

Nakon instalacije paketa, potrebno je pokrenuti Socket.io poslužitelj. Najprije se u Node.js-u kreira klasični HTTP poslužitelj. Zatim se objekt tog HTTP poslužitelja predaje kao argument funkciji za kreiranje Socket.io poslužitelja. Razlog tome je što Socket.io poslužitelj zahtijeva HTTP vezu, kako bi se provelo početno rukovanje i uspostavila željena WebSocket veza.

Nakon toga se HTTP poslužitelj pokreće na željenim vratima (engl. *port*) na uobičajeni način, metodom `listen()`. Navedeno se ostvaruje na način prikazan u sljedećem odsječku koda:

```
// ... potrebne require funkcije
const http = require("http");
const { Server } = require("socket.io");

const server = http.createServer();
const io = new Server(server);

const PORT = 3000;
server.listen(PORT, () => {console.log("Server pokrenut na portu 3000!");});
```

Nakon pokretanja poslužitelja, klijent se može povezati na njega. U kodu klijentske strane, to se postiže naredbom :

```
const socket = io();
```

Na poslužiteljskoj strani, u ovom trenutku moguće je definirati slušatelje događaja. Osnovni slušatelj događaja koji je potrebno implementirati odnosi se na događaj *connection*, koji označava povezivanje klijenta na poslužitelj. Slušatelji događaja implementiraju se korištenjem metode `.on()`. Definiranje slušatelja za događaj *connection* moglo bi izgledati

ovako:

```
io.on("connection", (socket) => {  
    console.log("Klijent povezan");  
});
```

U ovom primjeru, argument funkcije strelice (engl. *arrow function*), `socket`, predstavlja konkretnog klijenta koji se povezao. S obzirom da je taj objekt dostupan samo unutar slušatelja događaja povezivanja, često se implementiraju ugniježđeni slušatelji koji reagiraju isključivo na događaje koji se odnose na taj specifični klijent. Naime, funkcija `on()` pozvana na objektu `socket`, definira upravo takvog slušatelja. Primjer toga je događaj *disconnect* te njegov slušatelj, kao što prikazuje sljedeći primjer:

```
io.on("connection", (socket) => {  
    console.log("Klijent povezan");  
  
    socket.on("disconnect", () => {  
        console.log("Klijent odspojen. Id klijenta: ", socket.id);  
    });  
});
```

Ugniježdivanje se također koristi za definiranje slušatelja koji obrađuju događaje primanja poruke/podataka od uređaja, registraciju uređaja i sl.

Osim navedenih uobičajenih događaja, kao što je već spomenuto, `Socket.io` podržava definiranje i slanje proizvoljnih događaja. Događaje može slati bilo koja strana komunikacije, bilo poslužitelj, bilo klijent. Dakle, svaka strana može biti i pošiljalac i primatelj događaja. Za navedene proizvoljno definirane događaje, također se mogu definirati slušatelji. Događaj se

šalje korištenjem metode `emit()`, a kome će poslani događaj stići, ovisi o objektu nad kojim se metoda poziva. Poruka se može slati samo jednom klijentu, što se postiže pozivom metode `emit()` nad objektom koji predstavlja tog klijenta (`socket` u prethodnim primjerima). Ako se ista metoda pozove nad objektom koji predstavlja cijeli poslužitelj (`io`), poruka će biti poslana svim povezanim klijentima. Ovo je tzv. operacija emitiranja (engl. *broadcast*). Metoda, kao prvi argument, prima naziv događaja, a kao drugi, podatke. Navedeno je prikazano u sljedećem odsječku koda:

```
io.on("connection", (socket) => {  
  console.log("Klijent povezan");  
  
  const podaci = {a: 1, b: 2};  
  socket.emit("dogadaj_za_jednog_klijenta", podaci);  
  io.emit("dogadaj_za_sve_klijente", podaci);  
  
  socket.on("disconnect", () => {  
    console.log("Klijent odspojen. Id klijenta: ", socket.id);  
  });  
});
```

Na klijentskoj strani, mogao bi postojati ovakav kod za prihvaćanje navedenih događaja:

```
//KLIJENTSKI KOD
```

```
const socket = io(); //socket ovdje predstavlja poslužitelj
```

```
//SLUŠATELJ DOGAĐAJA ZA SPECIFIČNOG KLIJENTA
```

```
socket.on("dogadaj_za_jednog_klijenta", (podaci) => { // KOD FUNKCIJE });
```

```
//SLUŠATELJ DOGAĐAJA ZA SVE KLIJENTE
```

```
socket.on("dogadaj_za_sve_klijente", (podaci) => { // KOD FUNKCIJE });
```

Ako je potrebno poslati događaj više različitih, točno određenih klijenata, moguće je kreirati sobu, smjestiti sve potrebne klijente u nju te poslati poruku toj sobi. Soba se automatski kreira dodavanjem prvog klijenta u nju, pozivom metode `join()` nad objektom klijenta kojeg se želi dodati. Metoda `join()` kao argument prima naziv sobe.

```
socket.join("soba_za_uplate");
```

Slanje poruke sobi provodi se metodom `emit()`, no pozvanom nad objektom `io.to("naziv_sobe")`. Dakle, metoda `to()` specificira sobu primatelja poruke.

```
const podaci_o_uplati = {iznos: 1000, pdv: 50};  
io.to("soba_za_uplate").emit("uplata", podaci_o_uplati);
```

S obzirom da su sobe isključivo implementacijski detalj poslužiteljske strane, klijent ne može svojevoljno ući u sobu. Klijenta u sobu dodaje isključivo poslužitelj.

C.3. Socket.io klijent u programskom jeziku Python

S obzirom na to da se, u sklopu ovog rada, Python koristi isključivo u implementaciji klijentske strane aplikacije, ovo poglavlje opisuje samo Socket.io klijent [32]. Kako bi se započeo rad s Python Socket.io klijentom, potrebno je instalirati potrebni paket, što se čini naredbom:

```
pip install "python-socketio[client]"
```

U samom programskom kodu, potrebno je preuzetu funkcionalnost uključiti koristeći naredbu:

```
import socketio
```

Socket.io klijent kreira se korištenjem konstruktora, a na poslužitelj se povezuje metodom `connect()`:

```
sio = socketio.Client()
sio.connect("http://localhost:3001")
```

Python Socket.io implementacija podržava sve osnovne funkcionalnosti kao i Node.js implementacija. Proizvoljni događaji se primaju definiranjem funkcije, iznad koje se navodi dekorator `@[ime_klijenta].event`, u ovom slučaju `@sio.event`. Time se navedena funkcija označava kao funkcija koja se poziva pri pristizanju istoimenog događaja s poslužitelja, tj. funkcija postaje slušatelj događaja. Korištenjem dekoratora `@[ime_klijenta].on("naziv_događaja")` iznad funkcije, navedena se funkcija također označava kao slušatelj događaja, no ime događaja se navodi u samom dekoratoru te ne mora odgovarati imenu funkcije. Pri primitku označenog događaja, ta se funkcija automatski poziva. Navedeno je prikazano u sljedećem primjeru :

```
@sio.event
def poslana_naredba():
    print("Primitljena naredba s poslužitelja.")
    # Izvrsi naredbu
```

```
@sio.on("potvrda_registracije")
def potvrda():
    print("Registracija uspjela.")
```

Klijent ručno šalje događaje poslužitelju koristeći funkciju `emit()`. Ta funkcija, baš kao i u Node.js implementaciji, prima naziv događaja te podatke koji se šalju kao argumente. Korištenje te funkcije izgleda ovako:

```
podaci_o_uplati = {"cijena": 1000,
                  "placeno": 800,
                  "dug": 200,
                  "kamatna_stopa": 0.05}

sio.emit("uplata_dugovanja", podaci_o_uplati)
```

Korištenjem metode `wait()`, klijent se stavlja u stanje čekanja, dok god je veza s poslužiteljem uspostavljena. U ovom stanju, klijent prima događaje i na njih odgovara prema prethodno definiranim funkcijama slušateljima. Iako je glavna dretva programa u ovom slučaju blokirana, moguće je koristiti višedretvenost i koristiti pozadinske dretve za obavljanje nekog drugog aktivnog posla.

```
sio.wait() #Čekanje na događaje
```

Navedene funkcionalnosti ostvaruju tzv. pristup temeljen na događajima (engl. *event-driven*). Socket.io dopušta i klasični pristup, korištenjem metode `receive()`, koja blokira izvođenje programa, čeka na događaj i po primitku prestaje blokirati. Ovakav pristup bolji je za slučajeve u kojima je potrebna veća kontrola nad trenutkom primanja događaja, no odskače od pristupa

temeljenog na događajima za koji je Socket.io specijaliziran. Naime, glavna karakteristika pristupa temeljenog na događajima je samostalno pristizanje događaja, bez potrebe za slanjem zahtjeva ili provjerom postojanja događaja, kao što to radi naredba `receive()`.